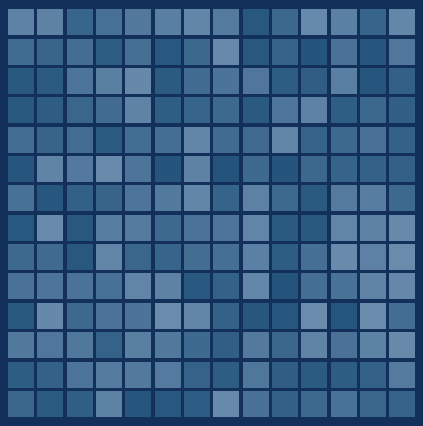


COMPUTER VISION FROM ZERO TO EXPERT

Python, Mathematics, Machine Learning, Deep Learning and Modern Computer Vision with PyTorch

A complete, practical, step-by-step course

GIRMAY GEBREMEDHIN ASSEFA

First Edition - 2026

Computer Vision From Zero to Expert

A Complete Practical Guide: Python, Mathematics, Machine Learning, Deep Learning and Modern Computer Vision with PyTorch

GIRMAY GEBREMEDHIN ASSEFA

First Edition, 2026

Copyright © 2026 Girmay Gebremedhin Assefa. All rights reserved.

Code examples are provided for learning. Library versions change; always consult the official documentation of NumPy, PyTorch, OpenCV, torchvision and the other libraries used in this book.

Contents

Preface 9

Who this book is for . . . 9
How the book is organised . . . 9
How to study with this book . . . 9
Conventions . . . 10
A note on software versions . . . 10

The Complete Roadmap at a Glance 11

Chapter 1: Getting Started: Your Environment and Python Basics 12

1.1 Why Python? . . . 12
1.2 Installing the tools . . . 12
1.3 Your first program . . . 13
1.4 Variables and data types . . . 13
1.5 Operators . . . 14
1.6 Conditionals . . . 14
1.7 Loops . . . 15
1.8 Putting it together: practice programs . . . 16
1.9 Common beginner mistakes . . . 16

Chapter 2: Data Structures, Functions, Files and Errors 18

2.1 Lists . . . 18
2.2 Tuples . . . 18
2.3 Dictionaries . . . 19
2.4 Sets . . . 19
2.5 Functions . . . 19
2.6 Reading and writing files . . . 20
2.7 Error handling . . . 21
2.8 Modules and the standard library . . . 21
2.9 Project: a to-do list application . . . 22

Chapter 3: Intermediate Python: Writing Code the Way ML Libraries Do 23

3.1 Comprehensions . . . 23
3.2 Lambda, map, filter, zip, enumerate . . . 23
3.3 Object-oriented programming . . . 23
3.4 Iterators and generators . . . 25
3.5 Decorators . . . 25

3.6 Context managers . . . 25
3.7 Type hints and good style . . . 25
3.8 Virtual environments and pip . . . 26
3.9 Git and GitHub . . . 26
3.10 Debugging and testing . . . 26
3.11 Project: an inventory manager (object-oriented) . . . 27

Chapter 4: The Scientific Python Stack: NumPy, Matplotlib, Pandas 28

4.1 Notebooks and Colab . . . 28
4.2 NumPy arrays . . . 28
4.3 Indexing and slicing . . . 29
4.4 Vectorisation and broadcasting . . . 29
4.5 Math, statistics and linear algebra with NumPy . . . 30
4.6 Matplotlib: visualisation . . . 30
4.7 Pandas: tabular data . . . 30
4.8 Images are arrays . . . 31
4.9 Project: image manipulation using only NumPy . . . 32

Chapter 5: Mathematics for Machine Learning 33

5.1 Linear algebra . . . 33
5.2 Calculus . . . 34
5.3 Optimisation and gradient descent . . . 35
5.4 Probability . . . 35
5.5 Statistics for machine learning . . . 36
5.6 Information theory essentials . . . 36
5.7 How to study the mathematics . . . 36

Chapter 6: Machine Learning Foundations 38

6.1 What is machine learning? . . . 38
6.2 The machine learning workflow . . . 38
6.3 Linear regression . . . 39
6.4 Logistic regression and classification . . . 39
6.5 Overfitting, underfitting and the bias-variance trade-off . . . 39
6.6 Regularisation . . . 40
6.7 Evaluation metrics . . . 40
6.8 Cross-validation and hyperparameter search . . . 41
6.9 Other classical models . . . 41
6.10 Feature engineering and why deep learning won . . . 42

Chapter 7: Neural Networks and Deep Learning Fundamentals 43

7.1 The artificial neuron . . . 43
7.2 Activation functions . . . 43
7.3 Layers and the multi-layer perceptron (MLP) . . . 44
7.4 The forward pass . . . 44
7.5 Loss functions . . . 44
7.6 Backpropagation . . . 44
7.7 A complete neural network from scratch . . . 44
7.8 Weight initialisation . . . 45
7.9 Vanishing and exploding gradients . . . 45
7.10 Hyperparameters of training . . . 46
7.11 Why deep learning works on images (and why MLPs are not enough) . . . 46

Chapter 8: PyTorch: The Deep Learning Framework 47

8.1 Installation and devices . . . 47
8.2 Tensors . . . 47
8.3 Autograd: automatic differentiation . . . 47
8.4 Building models with torch.nn . . . 48
8.5 Datasets and DataLoaders . . . 48
8.6 The training loop . . . 49
8.7 Saving and loading . . . 50
8.8 Using the GPU efficiently . . . 50
8.9 Reproducibility . . . 50
8.10 Debugging neural networks: a checklist . . . 50

Chapter 9: Training Deep Networks Well 52

9.1 Optimisers . . . 52
9.2 Learning-rate schedules . . . 52
9.3 Batch normalisation and other normalisation layers . . . 53
9.4 Regularisation techniques . . . 53
9.5 Data augmentation . . . 53
9.6 Dealing with class imbalance . . . 54
9.7 Hyperparameter tuning strategy . . . 54
9.8 Reading the training curves . . . 54
9.9 Practical recipe for a new vision problem . . . 55

Chapter 10: Classical Computer Vision with OpenCV 56

10.1 Images, pixels and colour spaces . . . 56
10.2 Basic operations . . . 56
10.3 Pixel-level processing . . . 57
10.4 Convolution and filtering . . . 57

10.5 Edge detection . . . 58
10.6 Morphological operations . . . 58
10.7 Contours and shape analysis . . . 58
10.8 Feature detection and matching . . . 58
10.9 Camera geometry and calibration . . . 59
10.10 Video and real-time processing . . . 59
10.11 Classical recognition: HOG + SVM . . . 59
10.12 Project: document scanner . . . 59

Chapter 11: Convolutional Neural Networks 61

11.1 Why convolutions? . . . 61
11.2 The convolution layer in detail . . . 61
11.3 Pooling . . . 62
11.4 Building a complete CNN . . . 62
11.5 Inspecting what a CNN learns . . . 63
11.6 Convolution variants you must know . . . 63
11.7 Dimensions cheat-sheet . . . 64

Chapter 12: Classic Architectures and Transfer Learning 65

12.1 The ImageNet era . . . 65
12.2 ResNet: the idea you must understand . . . 65
12.3 Transfer learning . . . 66
12.4 Choosing a backbone . . . 67
12.5 Model compression and efficiency . . . 67
12.6 Evaluation for classification . . . 68

Chapter 13: Object Detection 69

13.1 The problem and the vocabulary . . . 69
13.2 Evaluation: precision, recall, AP and mAP . . . 69
13.3 Datasets and annotation formats . . . 70
13.4 Two families of detectors . . . 70
13.5 Detection losses . . . 71
13.6 Hands-on: pretrained detection with torchvision . . . 71
13.7 Hands-on: training YOLO on your own data . . . 71
13.8 Fine-tuning torchvision Faster R-CNN on a custom dataset . . . 72
13.9 Practical issues . . . 72
13.10 Multi-object tracking . . . 72

Chapter 14: Image Segmentation 74

14.1 Three kinds of segmentation . . . 74

14.2 Data and metrics . . . 74
14.3 Loss functions . . . 74
14.4 Fully convolutional networks . . . 75
14.5 The encoder-decoder and U-Net . . . 75
14.6 Modern semantic segmentation architectures . . . 76
14.7 Instance segmentation . . . 76
14.8 Foundation models for segmentation: SAM . . . 76
14.9 Training pipeline example . . . 76
14.10 Special topics . . . 77

Chapter 15: Transformers, Self-Supervised Learning and Foundation Models 78

15.1 Attention . . . 78
15.2 The transformer block . . . 78
15.3 The Vision Transformer (ViT) . . . 79
15.4 Hierarchical and hybrid vision transformers . . . 79
15.5 Using pretrained vision transformers . . . 80
15.6 Self-supervised learning (SSL) . . . 80
15.7 Vision-language models . . . 80
15.8 The foundation-model workflow . . . 81
15.9 Scaling laws and compute . . . 81

Chapter 16: Generative Models for Vision 83

16.1 Autoencoders . . . 83
16.2 Variational autoencoders (VAE) . . . 83
16.3 Generative Adversarial Networks (GANs) . . . 83
16.4 Diffusion models . . . 84
16.5 Evaluating generative models . . . 85
16.6 Applications you can build . . . 85
16.7 Responsible use . . . 85
16.8 Neural rendering and 3D (awareness) . . . 85

Chapter 17: Advanced Vision Tasks and Applications 86

17.1 Keypoint detection and pose estimation . . . 86
17.2 Face analysis . . . 86
17.3 Optical character recognition and document understanding . . . 87
17.4 Video understanding . . . 87
17.5 Depth estimation, 3D vision and SLAM . . . 87
17.6 Anomaly detection and industrial inspection . . . 87
17.7 Medical imaging . . . 88
17.8 Remote sensing and geospatial vision . . . 88

17.9 Image retrieval, similarity and metric learning . . . 88
17.10 Image restoration and enhancement . . . 88
17.11 Robotics and embodied vision . . . 88
17.12 Autonomous driving perception stack . . . 89
17.13 Adversarial robustness and security . . . 89

Chapter 18: Data, Experimentation, Deployment and Engineering Practice 90

18.1 Data-centric AI . . . 90
18.2 Experiment management . . . 90
18.3 Error analysis . . . 91
18.4 Scaling training: multiple GPUs and efficiency . . . 91
18.5 Hugging Face and the open ecosystem . . . 91
18.6 Preparing a model for deployment . . . 91
18.7 Serving models . . . 92
18.8 Edge and mobile deployment . . . 93
18.9 Monitoring and maintenance (MLOps) . . . 93
18.10 Testing machine learning code . . . 93
18.11 Ethics, privacy and responsibility . . . 93

Chapter 19: Becoming an Expert: Projects, Research Skills and Career 94

19.1 The project ladder . . . 94
19.2 Competitions and community . . . 95
19.3 How to read a research paper . . . 95
19.4 Essential papers (your reading list) . . . 96
19.5 Reproducing and extending a paper . . . 96
19.6 Doing original research . . . 96
19.7 A twelve-month study schedule . . . 97
19.8 Courses and resources to complement this book . . . 98
19.9 Career paths . . . 98
19.10 Common mistakes of learners . . . 99
19.11 A final word . . . 99

Appendix A: Glossary 100

Appendix B: Quick Reference 102

B.1 Image tensor conventions . . . 102
B.2 Loss function chooser . . . 102
B.3 The ten-step debugging checklist . . . 102
B.4 Library installation cheat-sheet . . . 103
B.5 Where to go next . . . 103

Preface

Computer vision is the science of teaching machines to see. It is the technology that unlocks your phone with your face, lets a car read a stop sign, helps a doctor find a tumour in a scan, and allows a farmer to detect crop disease from a drone photograph. A decade ago these were research dreams. Today they are products, and the people who can build them are among the most sought-after engineers and researchers in the world.

This book has one purpose: to take you from the very first line of Python to the point where you can design, train, evaluate, debug, and deploy modern computer vision systems, and read research papers with understanding. It assumes **no prior knowledge** of programming, machine learning, or advanced mathematics. It does assume that you are willing to work: every chapter contains code you must type, run, break, and fix.

Who this book is for

- Complete beginners who want a structured path into computer vision and do not know where to start.
- Students preparing for graduate study or a PhD in computer vision, machine learning, or robotics, who need strong foundations and research habits.
- Software engineers and scientists from other fields who want to add deep learning for images to their skills.

How the book is organised

The book follows the roadmap of the learning path in five parts, each building on the last.

Part	Chapters	What you will master
Part I: Programming	1-4	Python from scratch, object-oriented programming, NumPy, Matplotlib, Pandas, Git
Part II: Mathematics and Machine Learning	5-6	Linear algebra, calculus, probability, classical ML with scikit-learn
Part III: Deep Learning	7-9	Neural networks from scratch, backpropagation, PyTorch, training techniques
Part IV: Computer Vision	10-17	Image processing, CNNs, architectures, detection, segmentation, transformers, generative models, advanced tasks
Part V: Professional Practice	18-19	Data, evaluation, deployment, research skills, projects, career

How to study with this book

1. **Type every example.** Do not copy and paste. Typing forces your brain to process each symbol, and errors you create yourself teach you the most.
2. **Predict before you run.** Before executing a cell, say aloud what you expect to see. When you are wrong, you have found something worth understanding.
3. **Do every exercise.** The exercises at the end of sections are the difference between having read about computer vision and being able to do it.
4. **Build alongside the book.** Each part ends with a project. Put the projects on GitHub. Your portfolio is your proof of ability.
5. **Be patient with the mathematics.** You do not need to master all mathematics before coding. Chapter 5 is designed to be read in parallel with the programming chapters.
6. **Use Google Colab.** Deep learning needs a GPU. Google Colab provides a free one in your browser, so you do not need an expensive computer.

> **TIP - A realistic schedule**
>
> At one to two hours a day, expect about nine to twelve months to finish this book properly. Consistency beats intensity: one hour every day is far better than ten hours once a week.

Conventions

Code appears in a grey box. Lines starting with `>>>` or comments such as `# output:` show what the program prints. Tip, note, warning and exercise boxes mark important side material. When a library function is introduced, its important arguments are explained, because understanding arguments is how you move from copying code to designing your own.

A note on software versions

The examples use Python 3.10 or newer, NumPy, Matplotlib, Pandas, scikit-learn, OpenCV (cv2), PyTorch, and torchvision. APIs evolve; if an example fails because of a version change, read the error message and the official documentation. Learning to do this is itself a core skill of an engineer.

The Complete Roadmap at a Glance

Before diving in, here is the complete map. Refer back to it whenever you feel lost.

Stage	Topics to master	Time
1. Python fundamentals	Variables, types, operators, conditionals, loops, lists/tuples/dicts/sets, functions, files, exceptions, modules	3-4 weeks
2. Intermediate Python	Comprehensions, lambda, OOP, generators, decorators, virtual environments, Git	3-4 weeks
3. Scientific Python	NumPy, Matplotlib, Pandas, Jupyter/Colab, image arrays	3-4 weeks
4. Mathematics	Linear algebra, calculus, probability, statistics, optimisation	6-8 weeks (parallel)
5. Machine learning	Regression, classification, metrics, overfitting, scikit-learn	4-6 weeks
6. Deep learning	Neurons, backpropagation, optimisers, PyTorch, regularisation	6-8 weeks
7. Computer vision	OpenCV, CNNs, architectures, transfer learning, detection, segmentation, ViT, generative models	8-12 weeks
8. Projects and research	Portfolio, Kaggle, experiment tracking, deployment, reading and reproducing papers	Ongoing

The rest of the book is the detailed walk through this table. Turn the page and begin.

CHAPTER 1

Getting Started: Your Environment and Python Basics

Every journey begins with preparation. In this chapter you will install the tools you need, run your first program, and learn the fundamental building blocks of Python: values, variables, operators, decisions, and repetition. By the end you will be able to write small but real programs.

1.1 Why Python?

Python is the language of machine learning. Nearly every deep learning library (PyTorch, TensorFlow, JAX), every image library (OpenCV, Pillow, scikit-image), and every research code repository you will read is written in Python. The reasons are practical:

- **Readable syntax.** Python code reads almost like English, so you spend your energy on ideas instead of punctuation.
- **A giant ecosystem.** Whatever you need, someone has already written a high-quality library.
- **Glue language.** Python is slow itself, but libraries such as NumPy and PyTorch run optimised C, C++ and CUDA code underneath. You write simple Python; the heavy computing runs at near hardware speed.

1.2 Installing the tools

You need three things: Python, a code editor, and a terminal.

Option A: Google Colab (zero installation)

Open colab.research.google.com with a Google account, create a new notebook, and you can run Python in your browser immediately. Colab also offers a free GPU (Runtime, then Change runtime type, then choose a GPU). Use it for deep learning chapters.

Option B: Local installation

1. Download Python 3.10 or newer from python.org. On Windows, tick **Add Python to PATH** during installation.
2. Install Visual Studio Code (VS Code) and its Python extension.
3. Open a terminal (Command Prompt, PowerShell, or Terminal) and check the installation.

```
python --version
pip --version
```

If `python` is not recognised on Linux or macOS, try `python3`. `pip` is Python's package installer: `pip install numpy` downloads and installs NumPy.

The terminal in five commands

You will use the terminal constantly. Learn these:

Command	Meaning
`pwd` (Windows: `cd`)	Show the current folder
`ls` (Windows: `dir`)	List files in the folder
`cd folder_name`	Enter a folder (`cd ..` goes up)
`mkdir name`	Create a folder
`python script.py`	Run a Python file

1.3 Your first program

Create a file named `hello.py` and type:

```
print("Hello, computer vision!")
```

Run it with `python hello.py`. The function `print()` displays text. The text inside quotes is a **string**. Congratulations: you are a programmer.

You can also use the **interactive interpreter**: type `python` in the terminal and enter expressions one at a time. It is perfect for experiments.

```
>>> 2 + 3
5
>>> "ab" * 3
'abababab'
```

1.4 Variables and data types

A **variable** is a name attached to a value. You create one with the assignment operator `=`.

```
age = 25                # int: whole number
height = 1.78           # float: decimal number
name = "Girmay"         # str: text
is_student = True       # bool: True or False
nothing = None          # NoneType: represents "no value"

print(type(age))        # <class 'int'>
print(type(height))     # <class 'float'>
```

Python is **dynamically typed**: you do not declare types; a variable takes the type of the value it holds, and may be reassigned to another type. Variable names should be descriptive, lowercase, with underscores (`learning_rate`, not `lr2x`). This style is called snake_case.

Numbers

```
a = 17
b = 5
print(a + b)    # 22  addition
print(a - b)    # 12  subtraction
print(a * b)    # 85  multiplication
print(a / b)    # 3.4 true division (always float)
print(a // b)   # 3   floor division
print(a % b)    # 2   remainder (modulo)
print(a ** b)   # 1419857  power
```

Floating-point numbers are stored in binary, so some decimals are inexact:

```
print(0.1 + 0.2)           # 0.30000000000000004
print(abs(0.1+0.2-0.3) < 1e-9)   # True  (compare floats with a tolerance)
```

This matters in machine learning: never test floats for exact equality; use a tolerance.

Strings

```
s = "Computer Vision"
print(len(s))            # 15
print(s.upper())         # COMPUTER VISION
print(s.lower())         # computer vision
print(s[0])              # C   (indexing starts at 0)
print(s[-1])             # n   (negative index counts from the end)
print(s[0:8])            # Computer  (slice: start inclusive, stop exclusive)
print(s.split(" "))      # ['Computer', 'Vision']
print(s.replace("Vision", "Science"))
print("Vision" in s)     # True
```

Strings are **immutable**: you cannot change a character in place; you create a new string.

f-strings: formatted output

An f-string embeds expressions inside curly braces. You will use them in every training loop to print losses.

```
epoch, loss, acc = 3, 0.48213, 0.9134
print(f"Epoch {epoch}: loss={loss:.3f}, accuracy={acc:.1%}")
# Epoch 3: loss=0.482, accuracy=91.3%
```

`:.3f` means three decimals; `:.1%` means percent with one decimal; `:>8` right-aligns in 8 characters; `:,` adds thousands separators.

Type conversion and input

```
x = int("42")          # 42
y = float("3.5")       # 3.5
z = str(100)           # "100"
n = int(input("Enter a number: "))   # input() always returns a string
```

1.5 Operators

Category	Operators	Example
Arithmetic	+ - * / // % **	7 // 2 is 3
Comparison	== != < > <= >=	3 != 4 is True
Logical	and, or, not	(x > 0) and (x < 10)
Membership	in, not in	"a" in "cat"
Identity	is, is not	x is None
Assignment	= += -= *= /=	x += 1 means x = x + 1

> **NOTE - == versus is**
>
> `==` compares values; `is` checks whether two names refer to the exact same object. Use `is` only for `None` (`if x is None`). Using `is` for numbers or strings is a classic bug.

1.6 Conditionals

Programs make decisions with `if`, `elif` and `else`. Python uses **indentation** (four spaces) to define blocks, not braces.

```
score = 82
if score >= 90:
    grade = "A"
elif score >= 80:
```

```
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
print(grade)   # B
```

Conditions can be chained: `0 < x < 10`. Python treats some values as false: `0`, `0.0`, `""`, `[]`, `{}`, `None`. Everything else is true. So `if my_list:` means "if the list is not empty".

A compact one-line form is the conditional expression:

```
label = "pass" if score >= 50 else "fail"
```

1.7 Loops

The for loop

A `for` loop repeats code for each item of a sequence.

```
for i in range(5):          # 0, 1, 2, 3, 4
    print(i, i ** 2)

for letter in "cat":
    print(letter)

for i in range(2, 10, 3):   # start, stop, step -> 2, 5, 8
    print(i)
```

The while loop

A `while` loop repeats as long as a condition is true. Training a neural network is, at heart, a loop.

```
total = 0
n = 1
while total < 100:
    total += n
    n += 1
print(n - 1, total)   # 14 105
```

Always make sure something inside the loop eventually makes the condition false, or you create an infinite loop (stop it with Ctrl+C).

break, continue, and else

```
for n in range(2, 30):
    for d in range(2, n):
        if n % d == 0:
            break            # leave the inner loop immediately
    else:
        print(n, "is prime") # runs only if the loop did NOT break

for i in range(10):
    if i % 2 == 0:
        continue             # skip to the next iteration
    print(i)                 # prints odd numbers
```

1.8 Putting it together: practice programs

Program 1: number guessing game

```
import random

secret = random.randint(1, 100)
attempts = 0
while True:
    guess = int(input("Guess the number (1-100): "))
    attempts += 1
    if guess < secret:
        print("Too low")
    elif guess > secret:
        print("Too high")
    else:
        print(f"Correct in {attempts} attempts!")
        break
```

Program 2: FizzBuzz

A classic interview exercise: print numbers 1 to 30, but "Fizz" for multiples of 3, "Buzz" for multiples of 5, and "FizzBuzz" for both.

```
for i in range(1, 31):
    if i % 15 == 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```

Program 3: temperature table

```
print(f"{'Celsius':>8} {'Fahrenheit':>11}")
for c in range(-10, 41, 10):
    f = c * 9 / 5 + 32
    print(f"{c:>8} {f:>11.1f}")
```

> **EXERCISE - Chapter 1 exercises**
>
> - Write a program that reads a number and prints whether it is even or odd.
> - Print the multiplication table of any number the user enters, from 1 to 12.
> - Write a program that sums the digits of an integer (hint: use `% 10` and `// 10` in a while loop).
> - Write a program that prints the first 20 Fibonacci numbers.
> - Write a program that checks whether a word entered by the user is a palindrome.

1.9 Common beginner mistakes

- **IndentationError:** mixing tabs and spaces, or forgetting to indent after a colon.
- **NameError:** using a variable before assigning it, or misspelling its name.
- **TypeError:** adding a string and a number (`"a" + 1`). Convert first.
- **Off-by-one errors:** `range(5)` gives 0 to 4, not 1 to 5.

> **TIP - Learn to read tracebacks**
>
> When Python crashes it prints a traceback. Read it from the bottom: the last line names the error and its cause; the lines above show where it happened. Copy the last line into a search engine; someone has solved it before.

CHAPTER 2

Data Structures, Functions, Files and Errors

Real programs manage collections of data and reuse logic. This chapter teaches Python's four built-in collections, how to write functions, how to read and write files, and how to handle errors without crashing. Everything here appears directly in computer vision code: lists of image paths, dictionaries of hyperparameters, functions for preprocessing, and files for datasets.

2.1 Lists

A **list** is an ordered, mutable collection. It is Python's workhorse.

```
nums = [10, 20, 30, 40]
nums.append(50)            # add to the end
nums.insert(1, 15)         # insert at index 1
nums.remove(30)            # remove first occurrence of value 30
last = nums.pop()          # remove and return the last item
print(nums)                # [10, 15, 20, 40]
print(nums[1:3])           # [15, 20]  slicing
print(nums[::-1])          # [40, 20, 15, 10]  reversed copy
print(len(nums), sum(nums), min(nums), max(nums))
nums.sort(reverse=True)    # sorts in place
print(sorted([3, 1, 2]))   # returns a new sorted list
```

Lists can hold mixed types and even other lists (a list of lists is a simple 2D grid, a primitive image):

```
image = [[0, 255, 0],
         [255, 0, 255],
         [0, 255, 0]]
print(image[1][2])         # 255  (row 1, column 2)
```

> **WARNING - Copying lists**
>
> `b = a` does not copy a list; both names refer to the same list. Use `b = a.copy()` or `b = a[:]` for a shallow copy, and `copy.deepcopy(a)` when the list contains other lists. Forgetting this causes some of the most confusing bugs in data code.

2.2 Tuples

A **tuple** is an ordered, immutable collection. Use tuples for fixed groupings, such as an image shape or coordinates.

```
shape = (480, 640, 3)      # height, width, channels
h, w, c = shape            # unpacking
point = (10, 20)
x, y = point
single = (5,)              # a one-element tuple needs the trailing comma
```

Functions that return several values actually return a tuple, and tuple unpacking is everywhere in ML code.

2.3 Dictionaries

A **dictionary** maps keys to values and provides very fast lookup.

```
config = {"lr": 0.001, "batch_size": 32, "epochs": 20}
print(config["lr"])               # 0.001
config["optimizer"] = "adam"      # add a key
print(config.get("momentum", 0.9))# default if key is missing
for key, value in config.items():
    print(key, "=", value)
print("epochs" in config)          # True (tests keys)
del config["epochs"]
print(list(config.keys()), list(config.values()))
```

Dictionaries hold hyperparameters, label mappings (`{0: "cat", 1: "dog"}`), and annotations. JSON files, the standard data-exchange format, map directly onto dictionaries and lists.

2.4 Sets

A **set** is an unordered collection of unique items, ideal for removing duplicates and for fast membership tests.

```
labels = ["cat", "dog", "cat", "bird", "dog"]
unique = set(labels)              # {'cat', 'dog', 'bird'}
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a | b)    # union {1,2,3,4,5}
print(a & b)    # intersection {3,4}
print(a - b)    # difference {1,2}
```

Splitting a dataset into train and test sets by image ID and checking that the two sets do not overlap (`train_ids & test_ids == set()`) is a real use that prevents data leakage.

2.5 Functions

A **function** packages reusable logic. It takes parameters and returns a result.

```
def normalize(pixels, max_value=255.0):
    """Scale a list of pixel values to the range 0-1."""
    return [p / max_value for p in pixels]

print(normalize([0, 128, 255]))    # [0.0, 0.50196..., 1.0]
```

Key ideas:

- **Parameters and arguments.** `pixels` and `max_value` are parameters; the values you pass are arguments. `max_value=255.0` is a **default argument**.
- **Keyword arguments.** Call `normalize([1, 2], max_value=2.0)`. Keywords make calls self-documenting; PyTorch functions use them heavily.
- **Docstring.** The triple-quoted string right after `def` documents the function. Always write one.
- **Return value.** Without `return`, a function returns `None`.
- **Scope.** Variables created inside a function are local. Avoid modifying global state.

*args and **kwargs

```
def describe(*args, **kwargs):
    print("positional:", args)       # a tuple
    print("keyword:", kwargs)        # a dict

describe(1, 2, lr=0.01, epochs=5)
# positional: (1, 2)
# keyword: {'lr': 0.01, 'epochs': 5}
```

You will meet `**kwargs` whenever a library function forwards options to another function. Similarly `f(*my_list)` unpacks a list into arguments.

Functions are values

Functions can be passed to other functions, stored in lists, and returned. This idea powers decorators and callback systems used in training frameworks.

```
def apply(func, data):
    return [func(x) for x in data]

print(apply(abs, [-3, 2, -1]))    # [3, 2, 1]
```

Recursion

A function may call itself. Each call must move toward a base case.

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

2.6 Reading and writing files

```
# Writing
with open("notes.txt", "w") as f:       # "w" truncates; "a" appends
    f.write("line one\n")
    f.write("line two\n")

# Reading
with open("notes.txt") as f:
    for line in f:                      # iterate line by line
        print(line.strip())

with open("notes.txt") as f:
    whole = f.read()                    # whole file as one string
```

The `with` statement guarantees the file is closed even if an error occurs. Always use it.

Paths with pathlib

```
from pathlib import Path

root = Path("dataset")
for p in sorted(root.glob("*/*.jpg")):  # e.g. dataset/cat/001.jpg
    label = p.parent.name               # folder name = class label
    print(p.name, label)
(root / "output").mkdir(exist_ok=True)  # create folder
```

`pathlib` works the same on Windows, macOS and Linux. This folder-per-class layout is exactly how image classification datasets are commonly organised.

JSON and CSV

```
import json, csv

data = {"model": "resnet18", "acc": 0.93}
with open("result.json", "w") as f:
    json.dump(data, f, indent=2)
with open("result.json") as f:
    loaded = json.load(f)

with open("labels.csv", newline="") as f:
    for row in csv.DictReader(f):       # each row is a dict
        print(row["filename"], row["label"])
```

2.7 Error handling

Errors are normal. The `try/except` statement lets you respond.

```
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print("Cannot divide by zero")
        return None

try:
    value = int("abc")
except ValueError as e:
    print("Bad number:", e)
else:
    print("Parsed", value)       # runs only if no exception
finally:
    print("always runs")         # cleanup code
```

Raise your own errors with `raise ValueError("learning rate must be positive")`. Catch **specific** exceptions; a bare `except:` hides real bugs.

> **TIP - Fail early and loudly**
>
> In ML code, validate inputs at the start: check image shapes, label ranges, and file existence. A clear error now is worth hours of debugging a mysterious wrong accuracy later.

2.8 Modules and the standard library

A **module** is a Python file you can import. A **package** is a folder of modules.

```
import math
from collections import Counter, defaultdict
import random, time, os, sys, datetime

print(math.sqrt(2), math.pi, math.exp(1))
print(Counter(["a", "b", "a", "c", "a"]))     # Counter({'a': 3, 'b': 1, 'c': 1})
random.seed(42)                               # reproducible randomness
print(random.sample(range(100), 5))
start = time.time(); sum(range(10**6)); print(time.time() - start)
```

To create your own module, save functions in `utils.py`, then `from utils import normalize` in another file in the same folder. Useful standard modules: `argparse` (command-line options), `logging`, `itertools`, `functools`, `os`, `shutil`.

Command-line scripts

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--epochs", type=int, default=10)
parser.add_argument("--lr", type=float, default=1e-3)
args = parser.parse_args()
print(args.epochs, args.lr)
# python train.py --epochs 30 --lr 0.0003
```

Real training scripts are launched exactly like this.

2.9 Project: a to-do list application

```
import json
from pathlib import Path

FILE = Path("todo.json")

def load():
    return json.loads(FILE.read_text()) if FILE.exists() else []

def save(items):
    FILE.write_text(json.dumps(items, indent=2))

def main():
    items = load()
    while True:
        cmd = input("[a]dd, [l]ist, [d]one, [q]uit: ").strip().lower()
        if cmd == "a":
            items.append({"task": input("Task: "), "done": False})
        elif cmd == "l":
            for i, it in enumerate(items):
                mark = "x" if it["done"] else " "
                print(f"{i}. [{mark}] {it['task']}")
        elif cmd == "d":
            idx = int(input("Number: "))
            items[idx]["done"] = True
        elif cmd == "q":
            save(items)
            break

if __name__ == "__main__":
    main()
```

The line `if __name__ == "__main__":` runs `main()` only when the file is executed directly, not when it is imported. Use it in every script.

> **EXERCISE - Chapter 2 exercises**
>
> • Write a word-frequency counter: read a text file, count how often each word occurs, and print the ten most common.
>
> • Write a function `split_dataset(items, train_ratio)` that shuffles a list and returns train and test lists.
>
> • Given a folder of images in class subfolders, build a dictionary mapping each class name to the number of images, and save it as JSON.
>
> • Write a function that validates a config dictionary and raises a clear error for missing or invalid keys.

CHAPTER 3

Intermediate Python: Writing Code the Way ML Libraries Do

PyTorch models are classes. Data pipelines use generators. Libraries rely on decorators and comprehensions. Experiments live in virtual environments and Git repositories. This chapter closes the gap between beginner scripts and professional machine learning code.

3.1 Comprehensions

A **comprehension** builds a collection in one readable expression.

```
squares = [x ** 2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
lengths = {word: len(word) for word in ["cat", "horse", "owl"]}
unique_lengths = {len(w) for w in ["cat", "dog", "horse"]}
flat = [v for row in [[1, 2], [3, 4]] for v in row]    # [1, 2, 3, 4]
```

A typical use: `paths = [p for p in all_files if p.suffix.lower() in {".jpg", ".png"}]`. If a comprehension becomes hard to read, use an ordinary loop; clarity wins.

3.2 Lambda, map, filter, zip, enumerate

```
square = lambda x: x * x                      # anonymous one-line function
print(list(map(square, [1, 2, 3])))           # [1, 4, 9]
print(list(filter(lambda x: x > 1, [1, 2, 3])))   # [2, 3]

names = ["a", "b", "c"]
scores = [0.9, 0.8, 0.7]
for n, s in zip(names, scores):               # pair items from two lists
    print(n, s)
for i, n in enumerate(names, start=1):        # get an index too
    print(i, n)

people = [("Ann", 31), ("Bo", 25)]
people.sort(key=lambda p: p[1])               # sort by age
```

`zip` and `enumerate` are used in nearly every training loop, for example `for i, (images, labels) in enumerate(loader)`.

3.3 Object-oriented programming

Classes bundle data (attributes) and behaviour (methods). This is essential: every neural network you build in PyTorch is a class.

```
class Rectangle:
    def __init__(self, width, height):      # constructor
        self.width = width                   # attributes
        self.height = height

    def area(self):                          # method
        return self.width * self.height

    def __repr__(self):                      # how it prints
        return f"Rectangle({self.width}, {self.height})"

r = Rectangle(3, 4)
print(r.area(), r)         # 12 Rectangle(3, 4)
```

`self` refers to the particular object. `__init__` runs when the object is created.

Inheritance

A child class reuses and extends a parent class.

```
class Shape:
    def area(self):
        raise NotImplementedError

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14159 * self.r ** 2

for s in [Square(2), Circle(1)]:
    print(s.area())        # polymorphism: same call, different behaviour
```

When the child defines its own `__init__`, call `super().__init__()` to run the parent's. PyTorch models begin with exactly that:

```
class MyModel(SomeParent):
    def __init__(self):
        super().__init__()
```

Special (dunder) methods

Methods named with double underscores customise behaviour: `__len__` (used by `len(obj)`), `__getitem__` (used by `obj[i]`), `__call__` (makes an object callable like a function). A PyTorch `Dataset` is simply a class implementing `__len__` and `__getitem__`. A model is called like `model(x)` because of `__call__`.

```
class Dataset:
    def __init__(self, items):
        self.items = items
    def __len__(self):
        return len(self.items)
    def __getitem__(self, idx):
        return self.items[idx]

d = Dataset(["a", "b", "c"])
print(len(d), d[1])        # 3 b
```

Dataclasses

```
from dataclasses import dataclass

@dataclass
class Config:
    lr: float = 1e-3
    epochs: int = 10
    batch_size: int = 32

cfg = Config(epochs=30)
print(cfg)    # Config(lr=0.001, epochs=30, batch_size=32)
```

Dataclasses give you tidy configuration objects without boilerplate.

3.4 Iterators and generators

A **generator** produces values one at a time, on demand, without storing everything in memory. This is how large datasets are streamed.

```
def read_batches(items, batch_size):
    for i in range(0, len(items), batch_size):
        yield items[i:i + batch_size]       # pause and hand back a value

for batch in read_batches(list(range(10)), 4):
    print(batch)        # [0,1,2,3] [4,5,6,7] [8,9]

total = sum(x * x for x in range(10**6))    # generator expression, no list built
```

Imagine a dataset of one million high-resolution images: you can never hold them all in RAM, so you yield them batch by batch. PyTorch's `DataLoader` does this for you.

3.5 Decorators

A **decorator** wraps a function to add behaviour.

```
import time, functools

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time() - t0:.3f}s")
        return result
    return wrapper

@timer
def slow_sum(n):
    return sum(range(n))

slow_sum(10**7)
```

You will see decorators such as `@torch.no_grad()`, `@property`, `@staticmethod` and `@dataclass`. You only need to understand them and use them; writing your own is occasional.

3.6 Context managers

The `with` statement works for any object defining enter and exit behaviour. PyTorch uses it: `with torch.no_grad():` temporarily disables gradient tracking.

```
from contextlib import contextmanager

@contextmanager
def stopwatch(name):
    t0 = time.time()
    yield
    print(name, round(time.time() - t0, 3), "s")

with stopwatch("sum"):
    sum(range(10**7))
```

3.7 Type hints and good style

```
def resize_box(box: tuple[int, int, int, int], scale: float) -> tuple[int, int, int, int]:
    x1, y1, x2, y2 = box
    return (int(x1 * scale), int(y1 * scale), int(x2 * scale), int(y2 * scale))
```

Type hints do not change how code runs, but they document intent and help your editor catch mistakes. Follow PEP 8: four-space indents, snake_case functions, CapWords classes,

lines of reasonable length. Tools like `black` and `ruff` format and check code automatically.

3.8 Virtual environments and pip

Different projects need different library versions. A **virtual environment** is an isolated Python installation per project.

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install numpy matplotlib pandas
pip freeze > requirements.txt      # record exact versions
pip install -r requirements.txt    # recreate the environment elsewhere
```

With Conda: `conda create -n cv python=3.11`, `conda activate cv`. Always record your dependencies; "it worked on my machine" is the enemy of reproducible research.

3.9 Git and GitHub

Git records the history of your code so you can experiment safely and collaborate.

```
git init
git add .
git commit -m "Add data loading script"
git status
git log --oneline
git checkout -b experiment-augmentation     # new branch
git merge experiment-augmentation           # bring it into the current branch
git remote add origin https://github.com/you/repo.git
git push -u origin main
```

Create a `.gitignore` containing `.venv/`, `__pycache__/`, `data/`, `*.pt` so datasets and weights are not committed. Commit small, often, and with meaningful messages. GitHub is also your public portfolio.

3.10 Debugging and testing

- Read the traceback from the bottom up.
- Use `print()` for quick inspection, but learn the debugger: `breakpoint()` stops execution and lets you inspect variables (`p var`, `n` next, `c` continue).
- Use VS Code breakpoints for step-through debugging.
- Write small tests with `pytest`:

```
# test_utils.py
from utils import normalize

def test_normalize_range():
    out = normalize([0, 255])
    assert out == [0.0, 1.0]
```

Run `pytest`. In ML, a good first test is a **sanity check**: a model must be able to overfit a tiny dataset of eight images. If it cannot, there is a bug.

3.11 Project: an inventory manager (object-oriented)

```
from dataclasses import dataclass, field
import json

@dataclass
class Item:
    name: str
    price: float
    qty: int = 0

class Inventory:
    def __init__(self):
        self.items: dict[str, Item] = {}

    def add(self, name, price, qty):
        if name in self.items:
            self.items[name].qty += qty
        else:
            self.items[name] = Item(name, price, qty)

    def remove(self, name, qty):
        item = self.items[name]
        if qty > item.qty:
            raise ValueError("not enough stock")
        item.qty -= qty

    def total_value(self):
        return sum(i.price * i.qty for i in self.items.values())

    def save(self, path):
        with open(path, "w") as f:
            json.dump([vars(i) for i in self.items.values()], f, indent=2)

inv = Inventory()
inv.add("camera", 120.0, 5)
inv.add("lens", 80.0, 3)
inv.remove("camera", 2)
print(inv.total_value())    # 120*3 + 80*3 = 600.0
```

Push this project to GitHub with a README explaining how to run it. It is your first portfolio entry.

> **EXERCISE - Chapter 3 exercises**
>
> • Write a `BankAccount` class with deposit, withdraw (raising an error on overdraft) and a statement method; then a `SavingsAccount` subclass that adds interest.
>
> • Write a generator that yields all image paths under a folder, one at a time.
>
> • Write a decorator `retry(n)` that retries a failing function up to n times.
>
> • Create a virtual environment, install three libraries, and produce a `requirements.txt`.
>
> • Create a Git repository, make five commits on two branches, and merge them.

CHAPTER 4

The Scientific Python Stack: NumPy, Matplotlib, Pandas

Computer vision is applied numerical computing. An image is a grid of numbers; a dataset is a table of numbers; a model is a stack of numerical operations. NumPy is the foundation of all of it. PyTorch tensors, OpenCV images, and scikit-learn inputs are all NumPy-like arrays. **Master this chapter completely.**

4.1 Notebooks and Colab

A **Jupyter notebook** mixes code, output, plots and text in cells. Run a cell with Shift+Enter. The last expression in a cell prints automatically. Install with `pip install jupyterlab` and start with `jupyter lab`, or use Google Colab. Magics: `%timeit expr` measures speed; `!pip install pkg` runs shell commands.

4.2 NumPy arrays

```
import numpy as np

a = np.array([1, 2, 3, 4])
M = np.array([[1, 2, 3],
              [4, 5, 6]])
print(M.shape)     # (2, 3): 2 rows, 3 columns
print(M.ndim)      # 2
print(M.dtype)     # int64
print(M.size)      # 6
```

An array has a **shape** (size of each dimension), an **ndim** (number of dimensions or axes), and a **dtype** (the element type). Common dtypes: `uint8` (0-255, used for images), `float32` (used for deep learning), `float64`, `int64`, `bool`.

Creating arrays

```
np.zeros((3, 4))              # 3x4 of zeros
np.ones((2, 2), dtype=np.float32)
np.full((2, 3), 7)
np.eye(3)                     # identity matrix
np.arange(0, 10, 2)           # [0 2 4 6 8]
np.linspace(0, 1, 5)          # 5 evenly spaced values from 0 to 1
rng = np.random.default_rng(42)
rng.random((2, 3))            # uniform [0,1)
rng.normal(0, 1, (2, 3))      # Gaussian
rng.integers(0, 256, (4, 4))  # random integers
```

Reshaping

```
x = np.arange(12)
y = x.reshape(3, 4)           # same data, new shape
z = x.reshape(2, -1)          # -1 means "infer this dimension" -> (2, 6)
y.T                           # transpose (4, 3)
y.flatten()                   # back to 1D (copy)
x[np.newaxis, :]              # add an axis -> (1, 12)
x[:, None]                    # -> (12, 1)
```

Understanding shape manipulation is critical. Most deep learning bugs are shape bugs. Get into the habit of printing `.shape` constantly.

4.3 Indexing and slicing

```
A = np.arange(20).reshape(4, 5)
# [[ 0  1  2  3  4]
#  [ 5  6  7  8  9]
#  [10 11 12 13 14]
#  [15 16 17 18 19]]
A[1, 2]          # 7
A[0]             # first row
A[:, 0]          # first column [0 5 10 15]
A[1:3, 2:5]      # rows 1-2, columns 2-4
A[::2, ::2]      # every second row and column
A[-1, -1]        # 19
```

Boolean (mask) indexing

```
v = np.array([5, -2, 9, 0, -7])
mask = v > 0                 # [ True False  True False False]
print(v[mask])               # [5 9]
v[v < 0] = 0                 # replace negatives by zero
A[A % 2 == 0]                # all even elements
```

Masks are how you threshold images, select detections above a confidence score, and filter predictions.

Fancy indexing

```
idx = [0, 2, 3]
A[idx]                       # rows 0, 2, 3
np.argmax(v), np.argmin(v)   # position of max / min
np.where(v > 0, 1, -1)       # elementwise if/else
```

> **NOTE - Views versus copies**
>
> Slices are **views**: modifying them modifies the original array. Boolean and fancy indexing return copies. Use `.copy()` when you need an independent array.

4.4 Vectorisation and broadcasting

Python loops are slow. NumPy applies operations to whole arrays at once, in compiled code. This is called **vectorisation**, and it is typically 10 to 100 times faster.

```
x = np.arange(1_000_000)
%timeit [i * 2 for i in range(1_000_000)]   # roughly 60 ms
%timeit x * 2                               # roughly 1 ms
```

Broadcasting

Broadcasting lets arrays of different shapes combine. The rules: compare shapes from the right; dimensions are compatible if they are equal or one of them is 1; a dimension of 1 is stretched.

```
A = np.ones((3, 4))
row = np.array([1, 2, 3, 4])          # shape (4,)  -> acts like (1, 4)
col = np.array([[10], [20], [30]])    # shape (3, 1)
print((A + row).shape)                # (3, 4)
print((A + col).shape)                # (3, 4)
print((col + row).shape)              # (3, 4)  outer-sum
```

A key computer vision example: subtract the per-channel mean from an image of shape (H, W, 3) with `img - mean.reshape(1, 1, 3)`. This is exactly what image normalisation does.

4.5 Math, statistics and linear algebra with NumPy

```
a = np.array([[1., 2.], [3., 4.]])
a.sum()                  # 10.0
a.sum(axis=0)            # column sums [4. 6.]
a.sum(axis=1)            # row sums    [3. 7.]
a.mean(), a.std(), a.min(), a.max()
np.exp(a), np.log(a), np.sqrt(a), np.abs(a)
np.clip(a, 1.5, 3.5)     # limit values to a range

b = np.array([[0., 1.], [1., 0.]])
a @ b                    # matrix multiplication
a * b                    # elementwise multiplication (different!)
np.linalg.inv(a)         # inverse
np.linalg.det(a)         # determinant
w, v = np.linalg.eig(a)  # eigenvalues and eigenvectors
np.linalg.norm(a)        # Frobenius norm
```

The `axis` argument is a frequent source of confusion: `axis=0` collapses rows (operates down columns); `axis=1` collapses columns. Think: the axis you name is the one that disappears.

4.6 Matplotlib: visualisation

```
import matplotlib.pyplot as plt

x = np.linspace(0, 2 * np.pi, 200)
plt.figure(figsize=(6, 3))
plt.plot(x, np.sin(x), label="sin")
plt.plot(x, np.cos(x), "--", label="cos")
plt.xlabel("x"); plt.ylabel("value")
plt.title("Trigonometric functions")
plt.legend(); plt.grid(True)
plt.show()
```

Multiple panels and other plot types

```
fig, axes = plt.subplots(1, 3, figsize=(10, 3))
axes[0].hist(np.random.randn(1000), bins=30)
axes[1].scatter(np.random.rand(50), np.random.rand(50))
axes[2].bar(["cat", "dog", "bird"], [10, 7, 3])
for ax in axes: ax.set_title("plot")
plt.tight_layout(); plt.show()
```

Showing images

```
img = np.random.randint(0, 256, (32, 32, 3), dtype=np.uint8)
plt.imshow(img)                       # colour image (H, W, 3)
gray = img.mean(axis=2)
plt.imshow(gray, cmap="gray")         # grayscale needs a colormap
plt.axis("off")
plt.show()
```

You will use `plt.plot` for training curves in every project: loss and accuracy versus epoch.

4.7 Pandas: tabular data

```
import pandas as pd

df = pd.DataFrame({
    "file": ["a.jpg", "b.jpg", "c.jpg", "d.jpg"],
    "label": ["cat", "dog", "cat", "bird"],
    "width": [640, 800, 640, None],
    "height": [480, 600, 480, 300],
})
print(df.head())
print(df.shape, df.dtypes)
df.info(); df.describe()
```

Selecting and filtering

```
df["label"]                     # a column (Series)
df[["file", "label"]]           # several columns
df.loc[0]                       # row by label
df.iloc[0:2]                    # rows by position
df[df["label"] == "cat"]        # filter rows
df[(df.height > 400) & (df.label != "dog")]   # combine with & |, with parentheses
df["area"] = df["width"] * df["height"]       # new column
```

Grouping, missing values, files

```
df["label"].value_counts()                # class balance
df.groupby("label")["height"].mean()
df["width"] = df["width"].fillna(df["width"].median())
df = df.dropna()                          # or drop missing rows
df.sort_values("height", ascending=False)
df.to_csv("meta.csv", index=False)
df2 = pd.read_csv("meta.csv")
```

Pandas is used to inspect datasets, check class imbalance, merge annotation tables, and analyse experiment results.

4.8 Images are arrays

This is the most important idea in the book. A **grayscale image** is a 2D array of shape (H, W) where each value is brightness (0 = black, 255 = white for `uint8`). A **colour image** is a 3D array of shape (H, W, 3), with three **channels**: red, green, blue. A **pixel** is one position; a colour pixel holds three numbers.

```
from PIL import Image

img = np.array(Image.open("photo.jpg"))       # shape (H, W, 3), dtype uint8
print(img.shape, img.dtype, img.min(), img.max())

crop = img[100:300, 150:400]                  # crop: slice rows and columns
flipped = img[:, ::-1]                        # horizontal flip
upside = img[::-1]                            # vertical flip
red_only = img.copy(); red_only[..., 1:] = 0  # zero the green and blue channels
bright = np.clip(img.astype(np.int16) + 60, 0, 255).astype(np.uint8)
gray = (0.299 * img[..., 0] + 0.587 * img[..., 1] + 0.114 * img[..., 2])
inverted = 255 - img                          # negative
```

> **WARNING - uint8 overflow**
>
> `np.uint8` wraps around: `np.uint8(250) + np.uint8(10)` gives 4, not 260. Convert to a wider type (`int16`, `float32`) before arithmetic, then clip and convert back. This bug silently ruins images.

Preparing images for neural networks

```
x = img.astype(np.float32) / 255.0                      # scale to [0, 1]
mean = np.array([0.485, 0.456, 0.406], dtype=np.float32)  # ImageNet statistics
std = np.array([0.229, 0.224, 0.225], dtype=np.float32)
x = (x - mean) / std                                    # per-channel normalise
x = x.transpose(2, 0, 1)                                # (H,W,C) -> (C,H,W)
x = x[np.newaxis]                                       # add batch dim -> (1,C,H,W)
```

Remember this chain: **load, scale, normalise, reorder axes, add batch dimension**. Matplotlib, OpenCV and PIL use (H, W, C); PyTorch uses (C, H, W). OpenCV additionally stores colour as **BGR** instead of RGB.

4.9 Project: image manipulation using only NumPy

Load an image and write functions (no OpenCV allowed) to: flip horizontally; crop the centre; adjust brightness and contrast (`out = alpha * img + beta`); convert to grayscale; apply a simple 3x3 box blur by averaging neighbours; compute and plot the histogram of pixel intensities; stitch four variations into a 2x2 grid with Matplotlib.

```
def box_blur(gray, k=3):
    pad = k // 2
    p = np.pad(gray, pad, mode="edge").astype(np.float32)
    out = np.zeros_like(gray, dtype=np.float32)
    for dy in range(k):
        for dx in range(k):
            out += p[dy:dy + gray.shape[0], dx:dx + gray.shape[1]]
    return (out / (k * k)).astype(np.uint8)
```

Notice that the loops run over the small kernel (9 iterations), not over pixels. This shifted-slice pattern is a core vectorisation technique and a preview of convolution.

EXERCISE - Chapter 4 exercises

- Create a 8x8 checkerboard array of 0s and 1s without loops and display it.
- Given a random (100, 3) array of points, compute the distance matrix (100, 100) using broadcasting only.
- Implement min-max normalisation and z-score standardisation for a dataset column-wise.
- Load a CSV with Pandas, plot the class distribution as a bar chart, and report missing values.
- Rotate an image by 90 degrees using only `transpose` and flips.

CHAPTER 5

Mathematics for Machine Learning

You do not need to be a mathematician to build models, but you do need the right intuitions. This chapter teaches exactly the mathematics used in deep learning and computer vision: linear algebra, calculus, probability, statistics, and optimisation. Read it alongside the programming chapters and always connect each idea to code.

5.1 Linear algebra

Scalars, vectors, matrices, tensors

A **scalar** is one number. A **vector** is an ordered list of numbers (a point in space, or a list of features). A **matrix** is a 2D grid. A **tensor** generalises these to any number of dimensions. A batch of colour images is a 4D tensor (N, C, H, W).

```
v = np.array([3., 4.])
np.linalg.norm(v)            # 5.0, the length (L2 norm)
u = np.array([1., 2.])
v @ u                        # dot product = 3*1 + 4*2 = 11
```

The dot product

The dot product `a . b` is the sum of elementwise products. Geometrically, `a . b = |a||b|cos(theta)`, so it measures **similarity** of direction. The **cosine similarity** `a.b / (|a||b|)` is used to compare image embeddings: values near 1 mean very similar. Every neuron computes a dot product: `w . x + b`.

Matrix multiplication

For A of shape (m, n) and B of shape (n, p), the product C = AB has shape (m, p), and `C[i, j]` is the dot product of row i of A and column j of B. The inner dimensions must match.

```
A = np.array([[1, 2], [3, 4], [5, 6]])    # (3, 2)
B = np.array([[1, 0, 2], [0, 1, 3]])      # (2, 3)
print((A @ B).shape)                      # (3, 3)
```

A fully connected layer is just `Y = X @ W + b`: a batch of inputs X multiplied by a weight matrix W. Matrix multiplication is not commutative (AB is not BA), but it is associative.

Transpose, identity, inverse

The **transpose** A.T flips rows and columns. The **identity** matrix I leaves vectors unchanged. The **inverse** A^-1 satisfies A A^-1 = I and exists only for square, non-singular matrices. Solving `Ax = b` is better done with `np.linalg.solve(A, b)` than by inverting.

Matrices as transformations

A matrix transforms space: it can rotate, scale, shear and project. A 2D rotation by angle t is the matrix `[[cos t, -sin t], [sin t, cos t]]`. In computer vision, **affine** and **homography** transformations (Chapter 10) are matrices applied to pixel coordinates.

```
t = np.deg2rad(90)
R = np.array([[np.cos(t), -np.sin(t)], [np.sin(t), np.cos(t)]])
```

```
print(R @ np.array([1, 0]))     # approximately [0, 1]
```

Eigenvalues, eigenvectors and SVD

An eigenvector v of A satisfies `A v = lambda v`: A only stretches it by the eigenvalue lambda. Eigenvectors reveal the main directions of a transformation. **Principal Component Analysis (PCA)** finds the eigenvectors of the covariance matrix, giving directions of greatest variance, used for dimensionality reduction and visualisation. The **Singular Value Decomposition** `A = U S V^T` factorises any matrix and underlies PCA, image compression, and the pseudo-inverse.

```
U, S, Vt = np.linalg.svd(gray_image, full_matrices=False)
k = 20
approx = (U[:, :k] * S[:k]) @ Vt[:k]     # rank-20 compression of an image
```

Keeping only the largest k singular values gives a compressed image; plot it for k = 5, 20, 50 and observe quality improving.

Norms and distances

The L1 norm sums absolute values; the L2 norm is the Euclidean length; the L-infinity norm is the largest absolute value. L1 and L2 norms appear in loss functions (L1 loss, MSE) and regularisation (Lasso, Ridge, weight decay).

5.2 Calculus

Derivatives

The **derivative** of f at x measures how fast f changes when x changes slightly: the slope of the tangent line. If `f(x) = x^2` then `f'(x) = 2x`. Key rules:

Function	Derivative
x^n	n x^(n-1)
e^x	e^x
ln x	1/x
sin x, cos x	cos x, -sin x
Sum rule	(f+g)' = f' + g'
Product rule	(fg)' = f'g + fg'
Chain rule	d/dx f(g(x)) = f'(g(x)) g'(x)

A neural network's learning is driven by derivatives: they tell you how to change each weight to reduce the error. You can verify a derivative numerically:

```
def numeric_grad(f, x, h=1e-5):
    return (f(x + h) - f(x - h)) / (2 * h)

print(numeric_grad(lambda x: x ** 3, 2.0))   # approximately 12.0 = 3 * 2^2
```

Partial derivatives and gradients

When a function has several inputs, a **partial derivative** measures change with respect to one input while holding the others fixed. The **gradient** is the vector of all partial derivatives: `grad f = [df/dx1, df/dx2, ...]`. It points in the direction of steepest increase. To minimise a function, step in the **opposite** direction.

The chain rule and backpropagation

The chain rule says that for composed functions, derivatives multiply. A network is a long composition: `loss = L(f3(f2(f1(x))))`. **Backpropagation** is nothing more than applying the chain rule repeatedly from the loss backwards, reusing intermediate results. If `y = w x`, `z = (y - t)^2`, then `dz/dw = dz/dy * dy/dw = 2(y - t) * x`. Chapter 7 builds this by hand.

The Jacobian and Hessian (awareness level)

The **Jacobian** is the matrix of first partial derivatives of a vector-valued function; the **Hessian** is the matrix of second derivatives, describing curvature. Deep learning mostly uses only first derivatives (gradients) because second-order methods are expensive, but curvature explains why optimisers behave as they do.

5.3 Optimisation and gradient descent

Training a model means finding parameters that minimise a **loss function**. **Gradient descent** does it iteratively:

```
theta = theta - lr * grad L(theta)
```

where `lr` is the **learning rate**.

```
# minimise f(x) = (x - 3)^2 + 1
x, lr = 10.0, 0.1
for step in range(50):
    grad = 2 * (x - 3)
    x = x - lr * grad
print(x)        # approximately 3.0
```

Experiment: lr = 1.1 makes the iteration diverge (steps overshoot); lr = 0.001 is painfully slow. Choosing the learning rate is the most important hyperparameter decision in deep learning.

Variants you will meet: **batch** gradient descent (uses all data per step), **stochastic** gradient descent (SGD, one example), and **mini-batch** SGD (a small batch, the standard). Real loss surfaces are non-convex, with many local minima, saddle points, and flat regions; in practice, deep networks still train well with SGD and its variants (momentum, Adam; Chapter 9).

5.4 Probability

Probability quantifies uncertainty. A **random variable** takes values with certain probabilities. Rules you need:

- Probabilities lie between 0 and 1 and sum to 1 over all outcomes.
- **Conditional probability:** `P(A|B) = P(A and B) / P(B)`.
- **Independence:** `P(A and B) = P(A) P(B)`.
- **Bayes' theorem:** `P(A|B) = P(B|A) P(A) / P(B)`.

Bayes' theorem in practice

Suppose 1 percent of images contain a defect, a detector catches 95 percent of real defects, and falsely flags 5 percent of good images. If it flags an image, what is the chance of a real defect?

```
p_def = 0.01
p_flag_given_def = 0.95
p_flag_given_ok = 0.05
p_flag = p_flag_given_def * p_def + p_flag_given_ok * (1 - p_def)
print(p_flag_given_def * p_def / p_flag)    # about 0.161
```

Only 16 percent, despite a "95 percent accurate" detector. This is why **precision** and class imbalance matter (Chapter 6).

Distributions

- **Bernoulli / Binomial:** yes-no outcomes (coin flip, classification).
- **Categorical:** one of K classes. The softmax output of a classifier is a categorical distribution.
- **Uniform:** all values equally likely (used for random initialisation).
- **Gaussian (normal):** the bell curve, parameterised by mean mu and variance sigma^2. Noise, weight initialisation and the VAE latent space use it.

Expectation, variance, covariance

The **expectation** (mean) is the probability-weighted average. **Variance** `E[(X - mu)^2]` measures spread; **standard deviation** is its square root. **Covariance** measures how two variables vary together; **correlation** is covariance normalised to [-1, 1].

5.5 Statistics for machine learning

- **Sample mean and variance** estimate population values from data.
- **Law of large numbers:** averages of many samples converge to the true mean; this is why mini-batch gradients are good estimates.
- **Central limit theorem:** sums of many independent variables look Gaussian.
- **Maximum likelihood estimation (MLE):** choose parameters that make the observed data most probable. Minimising cross-entropy loss is MLE for classification; minimising mean squared error is MLE under Gaussian noise.
- **Overfitting and confidence:** always report the variance across runs (different random seeds), not a single number.
- **Hypothesis testing and confidence intervals:** tell you whether an improvement of 0.3 percent accuracy is real or noise.

5.6 Information theory essentials

The **entropy** of a distribution measures its uncertainty: `H(p) = -sum p_i log p_i`. **Cross-entropy** between a true distribution p and a prediction q is `H(p, q) = -sum p_i log q_i`; it is the standard classification loss. **KL divergence** `KL(p||q) = sum p_i log(p_i/q_i)` measures how different q is from p; it appears in VAEs and knowledge distillation.

```
def cross_entropy(p, q):
    return -np.sum(p * np.log(q + 1e-12))

true = np.array([0, 1, 0])
print(cross_entropy(true, np.array([0.1, 0.8, 0.1])))    # 0.223 (good)
print(cross_entropy(true, np.array([0.6, 0.1, 0.3])))    # 2.303 (bad)
```

5.7 How to study the mathematics

- Watch 3Blue1Brown's *Essence of Linear Algebra* and *Essence of Calculus*: they build visual intuition no textbook can.
- Practise on Khan Academy; then implement every concept in NumPy.
- Always ask: where does this appear in a neural network?

Concept	Where it appears
Dot product	Every neuron; attention; cosine similarity
Matrix multiplication	Fully connected layers; convolution as matmul; transformers
Gradient, chain rule	Backpropagation
Gaussian distribution	Initialisation, noise, VAEs, diffusion models
Cross-entropy, KL	Classification loss, distillation, VAEs
Eigen/SVD	PCA, compression, analysing weights

EXERCISE - Chapter 5 exercises

• Write gradient descent in NumPy for `f(x, y) = x^2 + 3y^2` and plot the path on a contour plot.

• Implement cosine similarity and find the most similar pair among 10 random vectors.

• Compress a grayscale image using SVD with different ranks and plot reconstruction error versus rank.

• Verify numerically that the chain rule gives the same gradient as finite differences for `f(w) = (sigmoid(w x) - t)^2`.

• Simulate the defect-detection Bayes example with 100,000 random images and confirm the 16 percent figure.

CHAPTER 6

Machine Learning Foundations

Deep learning is a branch of machine learning. Almost every concept that guides deep learning practice (generalisation, overfitting, validation, metrics, regularisation) is best learnt on simple models first. This chapter gives you those foundations using scikit-learn.

6.1 What is machine learning?

Traditional programming: you write rules, the computer applies them to data and produces answers. Machine learning: you provide data and answers, and the computer **learns the rules**. The learnt rules are called a **model**.

Type	Data	Goal	Example
Supervised	Inputs with labels	Predict labels	Classify images as cat or dog
Unsupervised	Inputs only	Find structure	Cluster similar images
Self-supervised	Inputs; labels made from the data	Learn representations	Predict masked patches
Reinforcement	Interaction and rewards	Learn actions	Robot control

Supervised problems are of two kinds: **classification** (predict a category) and **regression** (predict a number, such as the age of a person in a photograph).

6.2 The machine learning workflow

1. Define the problem and the metric of success.
2. Collect and inspect data.
3. Split into **training**, **validation** and **test** sets.
4. Choose a model and a loss function.
5. Train on the training set.
6. Tune on the validation set.
7. Evaluate **once** on the test set.
8. Deploy and monitor.

Why three splits?

The **training set** fits the parameters. The **validation set** guides choices (model type, hyperparameters). The **test set** estimates real-world performance and must stay untouched until the end; if you tune on it, your reported accuracy is optimistic. Typical split: 70/15/15 or 80/10/10.

```
from sklearn.model_selection import train_test_split

X_train, X_tmp, y_train, y_tmp = train_test_split(X, y, test_size=0.3,
                                                  random_state=42, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(X_tmp, y_tmp, test_size=0.5,
                                                random_state=42, stratify=y_tmp)
```

`stratify=y` keeps class proportions equal in each split. `random_state` makes the split reproducible.

> **WARNING - Data leakage**
>
> Leakage happens when information from the test set influences training: normalising with statistics from the whole dataset, near-duplicate images in both train and test, or frames of one video split across sets. Leakage gives wonderful validation scores and terrible real-world performance. In computer vision, split by **source** (patient, video, location), not by image.

6.3 Linear regression

Model: `y_hat = w . x + b`. We choose w and b to minimise the **mean squared error** `MSE = mean((y - y_hat)^2)`. Here it is by gradient descent in NumPy, to see how learning works:

```
import numpy as np
rng = np.random.default_rng(0)
X = rng.uniform(-3, 3, (200, 1))
y = 2.5 * X[:, 0] + 1.0 + rng.normal(0, 0.5, 200)

w, b, lr = 0.0, 0.0, 0.05
for epoch in range(200):
    pred = w * X[:, 0] + b
    err = pred - y
    dw = 2 * np.mean(err * X[:, 0])
    db = 2 * np.mean(err)
    w -= lr * dw
    b -= lr * db
print(w, b)       # approximately 2.5 and 1.0
```

And in scikit-learn:

```
from sklearn.linear_model import LinearRegression
model = LinearRegression().fit(X, y)
print(model.coef_, model.intercept_)
```

6.4 Logistic regression and classification

For classification, we convert a score into a probability with the **sigmoid** `sigma(z) = 1 / (1 + e^-z)`, which squashes any number into (0, 1). The model `p = sigma(w . x + b)` is trained with **binary cross-entropy**. For multiple classes, **softmax** converts a score vector into probabilities that sum to 1: `softmax(z)_i = e^(z_i) / sum_j e^(z_j)`.

```
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

digits = load_digits()                  # 1797 images of 8x8 handwritten digits
X, y = digits.data, digits.target       # X shape (1797, 64)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=0, stratify=y)
clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
clf.fit(Xtr, ytr)
print("test accuracy:", clf.score(Xte, yte))     # about 0.97
```

You have just built your first image classifier: each 8x8 image is flattened into 64 features. Its weakness is that flattening discards spatial structure; CNNs (Chapter 11) fix this.

6.5 Overfitting, underfitting and the bias-variance trade-off

- **Underfitting (high bias):** the model is too simple; both training and validation errors are high.
- **Overfitting (high variance):** the model memorises training data; training error is low but validation error is high.
- **Good fit:** both errors are low and close together.

Symptom	Likely cause	Remedies
High train error, high val error	Underfitting	Bigger model, more features, train longer, lower regularisation
Low train error, high val error	Overfitting	More data, augmentation, regularisation, simpler model, early stopping
Val error fluctuates wildly	Small or noisy validation set; high learning rate	More data, lower learning rate

A **learning curve** (error versus training set size or epochs) is your main diagnostic tool. Always plot training and validation curves together.

```
from sklearn.tree import DecisionTreeClassifier
for depth in [1, 3, 5, 10, None]:
    t = DecisionTreeClassifier(max_depth=depth, random_state=0).fit(Xtr, ytr)
    print(depth, round(t.score(Xtr, ytr), 3), round(t.score(Xte, yte), 3))
# deep trees reach 1.000 on train but lower on test: overfitting
```

6.6 Regularisation

Regularisation discourages overly complex models by penalising large weights.

- **L2 (Ridge, weight decay):** add `lambda * sum(w^2)` to the loss; shrinks weights smoothly.
- **L1 (Lasso):** add `lambda * sum(|w|)`; drives some weights to exactly zero (feature selection).
- **Early stopping**, **dropout**, and **data augmentation** are further forms used in deep learning (Chapter 9).

```
from sklearn.linear_model import Ridge, Lasso
Ridge(alpha=1.0).fit(Xtr, ytr)      # alpha is the regularisation strength
```

6.7 Evaluation metrics

Accuracy (fraction correct) is misleading with imbalanced classes: a model that always says "no defect" scores 99 percent when 1 percent of images have defects. Use the **confusion matrix**:

	Predicted positive	Predicted negative
Actual positive	True Positive (TP)	False Negative (FN)
Actual negative	False Positive (FP)	True Negative (TN)

- **Precision** = TP / (TP + FP): of what I flagged, how much was right?
- **Recall** (sensitivity) = TP / (TP + FN): of all real positives, how many did I find?
- **F1** = 2 P R / (P + R): the harmonic mean.

- **Specificity** = TN / (TN + FP).
- **ROC curve and AUC:** true-positive rate versus false-positive rate across thresholds; AUC of 0.5 is chance, 1.0 is perfect.
- **Precision-recall curve and average precision:** better for imbalanced data. It leads directly to **mAP** for object detection (Chapter 13).
- **Regression metrics:** MAE, MSE, RMSE, and R-squared.

```
from sklearn.metrics import classification_report, confusion_matrix
pred = clf.predict(Xte)
print(confusion_matrix(yte, pred))
print(classification_report(yte, pred))
```

Choose the metric by the cost of mistakes. In cancer screening, a missed tumour (false negative) is far worse than a false alarm, so optimise recall. In a spam filter, blocking a legitimate email (false positive) is worse, so optimise precision.

6.8 Cross-validation and hyperparameter search

K-fold cross-validation splits the training data into K parts, trains K times each holding out one part, and averages the scores: a more reliable estimate with limited data.

```
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.svm import SVC

print(cross_val_score(SVC(), Xtr, ytr, cv=5).mean())
grid = GridSearchCV(SVC(), {"C": [0.1, 1, 10], "gamma": ["scale", 0.01, 0.001]}, cv=5)
grid.fit(Xtr, ytr)
print(grid.best_params_, grid.best_score_)
```

Hyperparameters (learning rate, depth, regularisation strength) are set by you; **parameters** (weights) are learnt. Random search often beats grid search in high dimensions.

6.9 Other classical models

- **k-Nearest Neighbours:** classify by majority vote of the k closest training points; no training, slow at prediction.
- **Decision trees:** a sequence of if-else questions; interpretable but prone to overfitting.
- **Random forests and gradient boosting (XGBoost, LightGBM):** ensembles of trees; dominant on tabular data.
- **Support vector machines:** maximum-margin classifiers; the kernel trick handles non-linear boundaries.
- **k-means clustering:** unsupervised grouping; used to build visual vocabularies and quantise colours.
- **PCA:** dimension reduction; **t-SNE** and **UMAP** for visualising high-dimensional embeddings.

```
from sklearn.cluster import KMeans
pixels = img.reshape(-1, 3).astype(np.float32)
km = KMeans(n_clusters=4, n_init=10, random_state=0).fit(pixels)
segmented = km.cluster_centers_[km.labels_].reshape(img.shape).astype(np.uint8)
# colour quantisation: the image now uses only 4 colours
```

6.10 Feature engineering and why deep learning won

Before deep learning, computer vision engineers hand-designed features (HOG, SIFT, edge histograms) and fed them to SVMs. Deep learning replaced hand-crafted features with **learnt features**: networks discover edges, textures and object parts directly from pixels. Understanding this history helps you read older papers and appreciate why CNNs work.

EXERCISE - Chapter 6 exercises

- Train kNN, logistic regression, SVM, and random forest on the digits dataset; compare test accuracy and training time.
- Plot the confusion matrix and identify which digits are confused with each other; show example errors with Matplotlib.
- Build an intentionally overfitted model, then fix it with regularisation; plot learning curves before and after.
- Create an imbalanced dataset (95 percent / 5 percent) and show why accuracy misleads; compute precision, recall and F1.
- Take Andrew Ng's Machine Learning Specialization (free to audit) in parallel for deeper treatment.

CHAPTER 7

Neural Networks and Deep Learning Fundamentals

Now we enter the heart of the subject. You will build a neural network from scratch using only NumPy, including backpropagation, so that PyTorch will never feel like magic. Understanding this chapter deeply separates engineers who copy code from engineers who diagnose and invent.

7.1 The artificial neuron

A neuron computes a weighted sum of its inputs plus a bias, then applies a non-linear **activation function**:

```
output = f(w1 x1 + w2 x2 + ... + wn xn + b) = f(w . x + b)
```

7.2 Activation functions

Without non-linear activations, stacking layers collapses into one linear transformation. Activations give networks the power to model complex functions.

Name	Formula	Notes
Sigmoid	1 / (1 + e^-x)	Output in (0,1); saturates; used for binary outputs
Tanh	(e^x - e^-x)/(e^x + e^-x)	Output in (-1,1); zero-centred; saturates
ReLU	max(0, x)	Default for hidden layers; cheap; can "die"
Leaky ReLU	max(0.01x, x)	Avoids dead neurons
GELU / SiLU (Swish)	x *Phi(x) / x* sigmoid(x)	Smooth; used in transformers and modern CNNs
Softmax	e^xi / sum e^xj	Multi-class probabilities at the output

```
def sigmoid(z): return 1 / (1 + np.exp(-z))
def relu(z): return np.maximum(0, z)
def softmax(z):
    z = z - z.max(axis=1, keepdims=True)      # subtract max for numerical stability
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)
```

7.3 Layers and the multi-layer perceptron (MLP)

Neurons are arranged in layers: an input layer, one or more **hidden layers**, and an output layer. In a **fully connected (dense)** layer every neuron connects to every input. A layer with n inputs and m neurons holds a weight matrix W of shape (n, m) and bias b of shape (m,), computing `H = f(X W + b)`. A network with many hidden layers is called **deep**. The **Universal Approximation Theorem** states that even a single hidden layer with enough neurons can approximate any continuous function; depth makes this efficient in practice.

7.4 The forward pass

For a two-layer classifier with input X (N, D), hidden size H and K classes:

`Z1 = X W1 + b1; A1 = relu(Z1); Z2 = A1 W2 + b2; P = softmax(Z2)`

7.5 Loss functions

A loss measures how wrong predictions are; training minimises it.

- **Mean squared error (MSE)** for regression: `mean((y - y_hat)^2)`.
- **Mean absolute error (L1)**: more robust to outliers.
- **Cross-entropy** for classification: `-mean(log P[i, y_i])`, the negative log-probability assigned to the true class.
- **Binary cross-entropy** for two classes or multi-label problems.
- **Huber / smooth L1**: quadratic for small errors, linear for large; used in detection.

7.6 Backpropagation

To train, we need the gradient of the loss with respect to every weight. **Backpropagation** computes these efficiently by applying the chain rule from output to input, reusing the stored forward-pass values.

For softmax with cross-entropy, the gradient of the loss with respect to the pre-softmax scores has a beautifully simple form: `dZ2 = (P - Y_onehot) / N`. Then, propagating backwards:

- `dW2 = A1.T @ dZ2`, `db2 = sum(dZ2, axis=0)`
- `dA1 = dZ2 @ W2.T`
- `dZ1 = dA1 * (Z1 > 0)` (ReLU gate: gradient flows only where the input was positive)
- `dW1 = X.T @ dZ1`, `db1 = sum(dZ1, axis=0)`

Each step multiplies the incoming gradient by a local derivative: that is the chain rule.

7.7 A complete neural network from scratch

We classify the 8x8 digits with a network written only in NumPy.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

X, y = load_digits(return_X_y=True)
X = X / 16.0                                  # scale pixels to [0, 1]
```

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=0)

rng = np.random.default_rng(0)
D, H, K = 64, 64, 10
W1 = rng.normal(0, np.sqrt(2 / D), (D, H)); b1 = np.zeros(H)    # He initialisation
W2 = rng.normal(0, np.sqrt(2 / H), (H, K)); b2 = np.zeros(K)

def forward(X):
    Z1 = X @ W1 + b1
    A1 = np.maximum(0, Z1)
    Z2 = A1 @ W2 + b2
    Z2 -= Z2.max(axis=1, keepdims=True)
    P = np.exp(Z2); P /= P.sum(axis=1, keepdims=True)
    return Z1, A1, P

lr, batch = 0.1, 64
for epoch in range(60):
    perm = rng.permutation(len(Xtr))
    for i in range(0, len(Xtr), batch):
        idx = perm[i:i + batch]
        xb, yb = Xtr[idx], ytr[idx]
        Z1, A1, P = forward(xb)
        N = len(xb)
        dZ2 = P.copy(); dZ2[np.arange(N), yb] -= 1; dZ2 /= N
        dW2 = A1.T @ dZ2; db2 = dZ2.sum(axis=0)
        dZ1 = (dZ2 @ W2.T) * (Z1 > 0)
        dW1 = xb.T @ dZ1; db1 = dZ1.sum(axis=0)
        W1 -= lr * dW1; b1 -= lr * db1
        W2 -= lr * dW2; b2 -= lr * db2
    if epoch % 10 == 0:
        _, _, P = forward(Xtr)
        loss = -np.log(P[np.arange(len(ytr)), ytr] + 1e-12).mean()
        acc = (forward(Xte)[2].argmax(1) == yte).mean()
        print(f"epoch {epoch:2d} loss {loss:.4f} test acc {acc:.3f}")
# test accuracy reaches roughly 0.97
```

Study every line. The inner loop is the **training loop** that all deep learning follows: (1) forward pass, (2) compute loss, (3) backward pass, (4) update parameters.

Gradient checking

To verify your backprop, compare it to the numerical gradient from Chapter 5 for a few weights. A relative difference below 1e-5 means your gradients are right. This is the best debugging technique when implementing new layers.

7.8 Weight initialisation

Starting all weights at zero makes every neuron identical (symmetry never breaks). Starting too large or too small makes activations explode or vanish as depth grows. Use **Xavier/Glorot** initialisation for tanh/sigmoid (variance 2/(fan_in+fan_out)) and **He/Kaiming** initialisation for ReLU (variance 2/fan_in). Modern frameworks apply good defaults, but you should know why.

7.9 Vanishing and exploding gradients

In deep networks, gradients are products of many numbers. If they are mostly below 1 they shrink toward zero (**vanishing**: early layers stop learning); if above 1 they blow up (**exploding**). Remedies: ReLU-family activations, careful initialisation, **batch normalisation**, **residual connections** (ResNet), and **gradient clipping**. These are the reasons modern architectures look the way they do.

7.10 Hyperparameters of training

Hyperparameter	Effect
Learning rate	Most important. Too high: divergence. Too low: slow, stuck
Batch size	Larger: smoother gradients, more memory; smaller: noisier, regularising
Epochs	Passes over the dataset; use early stopping
Hidden units / depth	Model capacity
Regularisation strength	Controls overfitting

7.11 Why deep learning works on images (and why MLPs are not enough)

Flattening a 224x224x3 image gives 150,528 inputs; a first hidden layer of 1,000 neurons needs 150 million weights, and ignores the fact that nearby pixels are related and that an object can appear anywhere. We need architectures that exploit the **structure of images**: locality and translation invariance. That is the convolutional neural network, the subject of Chapter 11.

EXERCISE - Chapter 7 exercises

- Extend the NumPy network with a second hidden layer and L2 regularisation; compare test accuracy.
- Implement gradient checking and verify your backward pass.
- Add momentum to the update rule and plot the loss curves with and without it.
- Replace ReLU by sigmoid in a 6-layer network and observe the vanishing gradient by printing the gradient norm at each layer.
- Train on a tiny subset of 20 images and confirm that the network reaches 100 percent training accuracy (the overfit sanity check).

CHAPTER 8

PyTorch: The Deep Learning Framework

PyTorch is the dominant framework for computer vision research and increasingly for production. It does three things for you: it provides **tensors** (NumPy-like arrays that run on GPUs), **autograd** (automatic differentiation, so you never write backpropagation by hand again), and a library of **neural network building blocks**.

8.1 Installation and devices

```
pip install torch torchvision torchaudio
```

Check the exact command for your system (CPU or CUDA) at pytorch.org. In Colab, PyTorch is already installed.

```
import torch
print(torch.__version__)
print(torch.cuda.is_available())
device = "cuda" if torch.cuda.is_available() else "cpu"
```

8.2 Tensors

```
x = torch.tensor([[1., 2.], [3., 4.]])
print(x.shape, x.dtype, x.device)
z = torch.zeros(2, 3); o = torch.ones(2, 3)
r = torch.randn(3, 4)                    # standard normal
a = torch.arange(10); l = torch.linspace(0, 1, 5)
x.T; x.reshape(4); x.view(1, 4)          # views share memory
x @ x; x * x; x.sum(dim=0); x.mean()
x[:, 0]; x[x > 2]                        # indexing like NumPy
x.unsqueeze(0); x.squeeze()              # add / remove size-1 dimensions
x.permute(1, 0)                          # reorder dimensions
torch.cat([x, x], dim=0)                 # join along an existing dim
torch.stack([x, x], dim=0)               # join along a new dim
x_gpu = x.to(device)                     # move to GPU
x_np = x.numpy()                         # to NumPy (CPU only, shares memory)
t = torch.from_numpy(x_np)
```

Note the dim/axis naming: PyTorch uses `dim`. Remember the key difference from NumPy for images: PyTorch expects **(N, C, H, W)**, while images load as (H, W, C). Use `permute(2, 0, 1)` to convert.

> **WARNING - Tensors on different devices**
>
> A very common error: "Expected all tensors to be on the same device". Model and data must both be on the same device. Move each batch with `.to(device)` inside the loop.

8.3 Autograd: automatic differentiation

```
w = torch.tensor(2.0, requires_grad=True)
x = torch.tensor(3.0)
y = (w * x - 5) ** 2            # builds a computation graph
y.backward()                    # computes dy/dw
print(w.grad)                   # 2*(w*x-5)*x = 2*(1)*3 = 6.0
```

`requires_grad=True` tells PyTorch to track operations. `backward()` runs backpropagation. Gradients **accumulate** in `.grad` across calls, so you must zero them before every step (`optimizer.zero_grad()`). To stop tracking (for evaluation), use `with torch.no_grad():`; to cut a tensor from the graph, use `.detach()`.

8.4 Building models with torch.nn

```
import torch.nn as nn
import torch.nn.functional as F

class MLP(nn.Module):
    def __init__(self, in_dim=784, hidden=256, num_classes=10):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden)
        self.fc2 = nn.Linear(hidden, num_classes)
        self.drop = nn.Dropout(0.2)

    def forward(self, x):
        x = x.flatten(1)                # (N, 1, 28, 28) -> (N, 784)
        x = F.relu(self.fc1(x))
        x = self.drop(x)
        return self.fc2(x)              # raw scores (logits), no softmax

model = MLP().to(device)
print(model)
print(sum(p.numel() for p in model.parameters()), "parameters")
```

Rules: define layers in `__init__`, define the computation in `forward`, and call the model as `model(x)` (never `model.forward(x)`). The model outputs **logits**; `nn.CrossEntropyLoss` applies softmax internally, so do not add softmax before it. Alternative for simple stacks: `nn.Sequential(nn.Linear(...), nn.ReLU(), ...)`.

8.5 Datasets and DataLoaders

```
from torch.utils.data import Dataset, DataLoader
from torchvision import datasets, transforms

tfm = transforms.Compose([transforms.ToTensor(),
                          transforms.Normalize((0.1307,), (0.3081,))])
train_ds = datasets.MNIST("data", train=True, download=True, transform=tfm)
test_ds = datasets.MNIST("data", train=False, download=True, transform=tfm)

train_loader = DataLoader(train_ds, batch_size=128, shuffle=True, num_workers=2)
test_loader = DataLoader(test_ds, batch_size=256)
```

`ToTensor()` converts a PIL image or NumPy array (H, W, C, uint8) to a float tensor (C, H, W) scaled to [0, 1]. `num_workers` loads data in parallel background processes. Shuffle only the training data.

A custom dataset for your own images

```
from PIL import Image
from pathlib import Path

class FolderDataset(Dataset):
    def __init__(self, root, transform=None):
        root = Path(root)
        self.classes = sorted(d.name for d in root.iterdir() if d.is_dir())
        self.class_to_idx = {c: i for i, c in enumerate(self.classes)}
        self.samples = [(p, self.class_to_idx[p.parent.name])
                        for p in root.glob("*/*") if p.suffix.lower() in {".jpg", ".png"}]
        self.transform = transform

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, i):
        path, label = self.samples[i]
        img = Image.open(path).convert("RGB")
        if self.transform:
            img = self.transform(img)
        return img, label
```

(`torchvision.datasets.ImageFolder` does exactly this for you.)

8.6 The training loop

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

def train_one_epoch(model, loader):
    model.train()                    # enables dropout and batchnorm training mode
    total, correct, loss_sum = 0, 0, 0.0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()                    # 1. clear old gradients
        logits = model(images)                   # 2. forward
        loss = criterion(logits, labels)         # 3. loss
        loss.backward()                          # 4. backward
        optimizer.step()                         # 5. update weights
        loss_sum += loss.item() * len(labels)
        correct += (logits.argmax(1) == labels).sum().item()
        total += len(labels)
    return loss_sum / total, correct / total

@torch.no_grad()
def evaluate(model, loader):
    model.eval()                                 # disables dropout, fixes batchnorm
    total, correct, loss_sum = 0, 0, 0.0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        logits = model(images)
        loss_sum += criterion(logits, labels).item() * len(labels)
        correct += (logits.argmax(1) == labels).sum().item()
        total += len(labels)
    return loss_sum / total, correct / total

for epoch in range(5):
    tr = train_one_epoch(model, train_loader)
    te = evaluate(model, test_loader)
    print(f"epoch {epoch+1}: train loss {tr[0]:.3f} acc {tr[1]:.3f} | test acc {te[1]:.3f}")
```

Memorise the five-step rhythm: **zero_grad, forward, loss, backward, step**. Always call `model.train()` before training and `model.eval()` before evaluating; forgetting `eval()` is a classic cause of strange accuracy.

8.7 Saving and loading

```
torch.save(model.state_dict(), "mlp.pt")            # save weights only (recommended)
model2 = MLP().to(device)
model2.load_state_dict(torch.load("mlp.pt", map_location=device))
model2.eval()

# checkpoint: resume training later
torch.save({"epoch": epoch, "model": model.state_dict(),
            "optimizer": optimizer.state_dict()}, "ckpt.pt")
```

8.8 Using the GPU efficiently

- Move model and batches to the device; use `pin_memory=True` and several `num_workers` in the DataLoader.
- Use **mixed precision** to speed up and halve memory:

```
scaler = torch.cuda.amp.GradScaler()
for images, labels in train_loader:
    images, labels = images.to(device), labels.to(device)
    optimizer.zero_grad()
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        loss = criterion(model(images), labels)
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

- If you run out of memory (OOM): reduce batch size, use mixed precision, **gradient accumulation** (step every few batches), or gradient checkpointing.

8.9 Reproducibility

```
import random, numpy as np
def seed_everything(seed=42):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
```

Report results as mean and standard deviation over several seeds.

8.10 Debugging neural networks: a checklist

1. Print shapes at every stage; check one batch's image range and labels.
2. Overfit a single batch to near-zero loss. If you cannot, there is a bug.
3. The initial loss for K balanced classes should be about `ln(K)` (2.30 for 10 classes).
4. Visualise the images exactly as the model sees them (after augmentation and normalisation).
5. Check that labels match images, and that you call `zero_grad`, `train()`, `eval()` correctly.
6. Try a smaller learning rate; watch for NaN losses (often a too-high learning rate or log of zero).
7. Monitor gradient norms and weight statistics.

EXERCISE - Chapter 8 exercises

• Train the MLP on MNIST, then Fashion-MNIST; reach at least 97 percent and 88 percent respectively.

• Write a custom Dataset for a folder of your own images and plot a batch with its labels.

• Implement `fit(model, train_loader, val_loader, epochs)` that records history and plots loss and accuracy curves.

• Save the best model by validation accuracy and reload it in a new session.

• Re-implement the NumPy network of Chapter 7 in PyTorch and compare the results.

CHAPTER 9

Training Deep Networks Well

Getting a network to run is easy; getting it to train well is the craft. This chapter covers optimisers, learning-rate schedules, normalisation, regularisation, data augmentation, and systematic tuning: the practical knowledge that determines whether a project succeeds.

9.1 Optimisers

Optimiser	Idea	When to use
SGD	Step along the negative gradient	Strong baseline; best final accuracy with tuning
SGD + momentum	Accumulate a velocity to smooth and accelerate	Standard for CNNs (momentum 0.9)
Nesterov	Look-ahead momentum	Small improvement over momentum
RMSprop	Scale steps by running average of squared gradients	RNNs, some RL
Adam	Momentum plus per-parameter scaling	Default; fast and robust; lr about 1e-3
AdamW	Adam with decoupled weight decay	Default for transformers; lr about 1e-4 to 1e-3

Momentum update: `v = beta v + grad; theta = theta - lr v`. Adam keeps running means of the gradient (m) and squared gradient (v), and updates `theta -= lr * m_hat / (sqrt(v_hat) + eps)`.

```
opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9,
                      weight_decay=5e-4, nesterov=True)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)
```

Rule of thumb: start with AdamW at 1e-3 (or 3e-4) to get a quick working baseline; try SGD with momentum and a schedule when you want the best accuracy on CNNs.

9.2 Learning-rate schedules

The best learning rate changes during training: begin larger to make progress, end smaller to settle.

```
from torch.optim.lr_scheduler import StepLR, CosineAnnealingLR, OneCycleLR

sched = StepLR(opt, step_size=30, gamma=0.1)              # multiply by 0.1 every 30 epochs
sched = CosineAnnealingLR(opt, T_max=100)                 # smooth cosine decay
sched = OneCycleLR(opt, max_lr=0.1, total_steps=100 * len(train_loader))   # up then down
# call sched.step() after each epoch (or after each batch for OneCycleLR)
```

Warmup (starting with a tiny learning rate and ramping up over the first epochs) stabilises transformers and large-batch training. **Learning-rate range test**: increase the rate exponentially over one epoch while plotting loss; pick a value somewhat below where loss starts to explode.

9.3 Batch normalisation and other normalisation layers

Batch normalisation (BatchNorm) normalises each channel across the batch to zero mean and unit variance, then applies a learnable scale and shift. Benefits: faster training, higher learning rates, mild regularisation, and less sensitivity to initialisation. In PyTorch: `nn.BatchNorm2d(channels)` after a convolution. During training it uses batch statistics; during evaluation it uses running averages (hence `model.eval()` matters).

Alternatives: **LayerNorm** (normalises across features of one sample; used in transformers), **GroupNorm** (works with small batches, common in detection and segmentation), **InstanceNorm** (style transfer).

9.4 Regularisation techniques

- **Weight decay (L2):** keeps weights small. Typical 1e-4 to 5e-4 for CNNs with SGD, 0.01 to 0.05 with AdamW.
- **Dropout:** randomly zero activations (probability p) during training; prevents co-adaptation. Typical p of 0.2 to 0.5 in fully connected layers; rarely used inside modern CNN blocks.
- **Early stopping:** stop when validation loss stops improving for a patience of several epochs; keep the best checkpoint.
- **Data augmentation:** the single most effective regulariser in vision (next section).
- **Label smoothing:** soft targets (for example 0.9 instead of 1) reduce overconfidence: `nn.CrossEntropyLoss(label_smoothing=0.1)`.
- **Mixup and CutMix:** blend two images and their labels; strong regularisers for classification.
- **Stochastic depth / DropPath:** randomly skip residual blocks; used in modern architectures.

```
class EarlyStopping:
    def __init__(self, patience=7):
        self.patience, self.best, self.bad = patience, float("inf"), 0
    def step(self, val_loss):
        if val_loss < self.best - 1e-4:
            self.best, self.bad = val_loss, 0
            return False            # do not stop; this was an improvement
        self.bad += 1
        return self.bad >= self.patience
```

9.5 Data augmentation

Augmentation creates new training examples by label-preserving transformations. It teaches the network invariances and multiplies the effective dataset size.

```
from torchvision import transforms as T

train_tfm = T.Compose([
    T.RandomResizedCrop(224, scale=(0.6, 1.0)),
    T.RandomHorizontalFlip(),
    T.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3, hue=0.05),
    T.RandomRotation(15),
    T.ToTensor(),
    T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
    T.RandomErasing(p=0.25),
])
val_tfm = T.Compose([T.Resize(256), T.CenterCrop(224), T.ToTensor(),
```

```
    T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])
```

Important principles:

- **Augment only the training set.** Validation and test use deterministic preprocessing.
- **Augmentations must preserve the label.** Horizontal flipping is fine for cats but wrong for digit recognition (6 versus 9 with vertical flips) or for text.
- **For detection and segmentation, transform boxes and masks together with the image.** Use libraries such as **Albumentations** or torchvision's v2 transforms.
- Modern recipes: **AutoAugment, RandAugment, TrivialAugment, Mixup, CutMix**.

9.6 Dealing with class imbalance

- Use class weights in the loss: `nn.CrossEntropyLoss(weight=w)`.
- Oversample minority classes with `WeightedRandomSampler`, or undersample majority classes.
- Use **focal loss**, which down-weights easy examples (used in detection).
- Evaluate with precision, recall, F1 or AUC rather than accuracy.

9.7 Hyperparameter tuning strategy

1. Get a baseline running end to end on a small subset.
2. Tune the **learning rate** first, then batch size, weight decay, augmentation strength, and architecture size.
3. Change **one thing at a time** and log everything.
4. Prefer **random search** to grid search; sample the learning rate on a log scale.
5. Use tools: **Weights & Biases**, **TensorBoard**, **Optuna**.

```
# TensorBoard logging
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter("runs/exp1")
writer.add_scalar("loss/train", train_loss, epoch)
writer.add_scalar("acc/val", val_acc, epoch)
```

9.8 Reading the training curves

Observation	Diagnosis	Action
Loss does not decrease at all	Learning rate far off, bug, wrong labels	Overfit one batch; check pipeline
Loss becomes NaN	Learning rate too high, log(0), exploding gradients	Lower lr, clip gradients, add eps
Train acc high, val acc low	Overfitting	More augmentation, regularisation, data
Both low and plateau	Underfitting	Bigger model, longer training, higher lr
Val loss rising while val acc rising	Overconfidence, calibration	Label smoothing; judge by the metric you care about
Spiky loss	Learning rate or batch too small	Lower lr, warmup, gradient clipping

Gradient clipping: `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` before `optimizer.step()`.

9.9 Practical recipe for a new vision problem

1. Inspect 50 random images and labels. Check class balance and image sizes.
2. Start from a **pretrained** model (Chapter 12); do not train from scratch unless you have a huge dataset.
3. Train a baseline with AdamW, lr 1e-3 for the new head and 1e-4 for the backbone, cosine schedule, light augmentation.
4. Look at failures (the misclassified images) to guide improvements; most gains come from data.
5. Scale up the model and the resolution only after the data pipeline is solid.

EXERCISE - Chapter 9 exercises

- Train a small CNN on CIFAR-10 with and without augmentation; plot the validation curves.
- Implement a learning-rate range test and pick a learning rate from the plot.
- Compare SGD+momentum, Adam and AdamW at equal budget on the same model.
- Implement Mixup (blend two images and labels with a Beta-distributed weight) and measure its effect.
- Log three experiments to TensorBoard or Weights & Biases and compare them.

CHAPTER 10

Classical Computer Vision with OpenCV

Before neural networks dominated, computer vision was built from mathematical image operations. These techniques are still used everywhere: for preprocessing, augmentation, data cleaning, real-time systems with limited compute, camera geometry, and as building blocks inside deep models. A real expert knows both worlds.

10.1 Images, pixels and colour spaces

OpenCV (`cv2`) is the standard library: `pip install opencv-python`.

```
import cv2, numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("photo.jpg")            # BGR order, uint8, shape (H, W, 3)
print(img.shape, img.dtype)
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)   # convert before showing with Matplotlib
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.imwrite("gray.png", gray)
plt.imshow(rgb); plt.axis("off"); plt.show()
```

> **WARNING - BGR versus RGB**
>
> OpenCV loads colour images as **BGR**, not RGB. If you show an OpenCV image with Matplotlib or feed it to a pretrained PyTorch model without converting, colours are swapped and accuracy drops. Convert with `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.

Colour spaces

- **RGB/BGR:** additive channels; mixes brightness and colour.
- **HSV:** Hue (colour type), Saturation (purity), Value (brightness). Excellent for colour-based segmentation because brightness is separated.
- **LAB:** perceptually uniform; L is lightness. Used for colour correction.
- **Grayscale:** one channel; many algorithms start here.

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)       # H in [0,179], S,V in [0,255]
lower, upper = np.array([35, 80, 50]), np.array([85, 255, 255])    # green range
mask = cv2.inRange(hsv, lower, upper)            # 255 where pixel is in range
green_only = cv2.bitwise_and(img, img, mask=mask)
```

10.2 Basic operations

```
h, w = img.shape[:2]
small = cv2.resize(img, (w // 2, h // 2), interpolation=cv2.INTER_AREA)  # (width, height)!
big = cv2.resize(img, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)
crop = img[50:250, 100:400]
flip = cv2.flip(img, 1)                           # 1 horizontal, 0 vertical
M = cv2.getRotationMatrix2D((w / 2, h / 2), 30, 1.0)   # centre, angle, scale
rot = cv2.warpAffine(img, M, (w, h))
cv2.rectangle(img, (50, 50), (200, 150), (0, 255, 0), 2)
cv2.circle(img, (120, 100), 30, (0, 0, 255), -1)
cv2.putText(img, "cat", (50, 40), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
```

Note that OpenCV sizes are **(width, height)** while NumPy shapes are **(height, width)**. Use `INTER_AREA` when shrinking and `INTER_LINEAR` or `INTER_CUBIC` when enlarging.

10.3 Pixel-level processing

Brightness, contrast, gamma

`out = alpha * img + beta` adjusts contrast and brightness: `cv2.convertScaleAbs(img, alpha=1.3, beta=20)`. **Gamma correction** applies a non-linear curve: `out = 255 * (img/255) ** gamma`, brightening dark regions for gamma below 1.

Histograms and equalisation

An intensity **histogram** counts pixels at each level. **Histogram equalisation** spreads intensities to enhance contrast; **CLAHE** does it locally and avoids over-amplifying noise.

```
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])
eq = cv2.equalizeHist(gray)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(gray)
```

Thresholding

Thresholding converts grayscale to binary: pixels above a threshold become white.

```
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
_, otsu = cv2.threshold(gray, 0, 255,
                        cv2.THRESH_BINARY + cv2.THRESH_OTSU)   # automatic threshold
adaptive = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                 cv2.THRESH_BINARY, 11, 2)           # for uneven lighting
```

10.4 Convolution and filtering

Convolution slides a small matrix (a **kernel** or filter) over the image; at each position, it multiplies overlapping values and sums them. It is the central operation of both classical vision and CNNs. Different kernels produce different effects.

```
def convolve2d(img, kernel):
    kh, kw = kernel.shape
    pad_h, pad_w = kh // 2, kw // 2
    padded = np.pad(img, ((pad_h, pad_h), (pad_w, pad_w)),
                    mode="reflect").astype(np.float32)
    out = np.zeros(img.shape, np.float32)
    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            out[i, j] = np.sum(padded[i:i + kh, j:j + kw] * kernel)
    return out
```

(Strictly, this computes cross-correlation, which is what CNN libraries also call convolution.) In practice, use `cv2.filter2D(gray, -1, kernel)`.

Smoothing (denoising) filters

```
box = cv2.blur(gray, (5, 5))                       # mean filter
gauss = cv2.GaussianBlur(gray, (5, 5), sigmaX=1.5) # weighted by a Gaussian; most common
median = cv2.medianBlur(gray, 5)                   # removes salt-and-pepper noise
bilateral = cv2.bilateralFilter(img, 9, 75, 75)    # smooths but preserves edges
```

Sharpening

```
sharpen = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]], np.float32)
sharp = cv2.filter2D(img, -1, sharpen)
```

10.5 Edge detection

Edges are sudden changes in intensity, corresponding to the image gradient. The **Sobel** operator estimates horizontal and vertical derivatives.

```
gx = cv2.Sobel(gray, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(gray, cv2.CV_32F, 0, 1, ksize=3)
magnitude = np.sqrt(gx ** 2 + gy ** 2)
direction = np.arctan2(gy, gx)
```

The **Canny** detector is the standard: it (1) smooths, (2) computes gradients, (3) thins edges with non-maximum suppression, and (4) links edges by hysteresis using two thresholds.

```
edges = cv2.Canny(cv2.GaussianBlur(gray, (5, 5), 1.4), 100, 200)
```

The first convolutional layers of a trained CNN learn filters that resemble these edge and colour detectors: a beautiful link between classical and deep vision.

10.6 Morphological operations

Used on binary images to clean masks. **Erosion** shrinks white regions; **dilation** grows them; **opening** (erode then dilate) removes small noise; **closing** (dilate then erode) fills small holes.

```
k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
opened = cv2.morphologyEx(otsu, cv2.MORPH_OPEN, k)
closed = cv2.morphologyEx(otsu, cv2.MORPH_CLOSE, k)
```

10.7 Contours and shape analysis

```
contours, _ = cv2.findContours(opened, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for c in contours:
    area = cv2.contourArea(c)
    if area < 200: continue
    x, y, w_, h_ = cv2.boundingRect(c)
    peri = cv2.arcLength(c, True)
    approx = cv2.approxPolyDP(c, 0.02 * peri, True)   # corner count gives the shape
    cv2.drawContours(img, [c], -1, (0, 255, 0), 2)
```

A complete classical pipeline for counting coins or cells: grayscale, blur, Otsu threshold, morphological cleaning, find contours, filter by area, count.

10.8 Feature detection and matching

To find the same point in two images, we need **keypoints** and **descriptors**: compact numerical summaries of local appearance, ideally invariant to scale, rotation and lighting.

- **Harris / Shi-Tomasi:** corner detectors.
- **SIFT:** scale and rotation invariant; accurate, robust.
- **ORB:** fast, free, binary descriptor; good for real-time.

```
orb = cv2.ORB_create(nfeatures=1000)
kp1, des1 = orb.detectAndCompute(gray1, None)
kp2, des2 = orb.detectAndCompute(gray2, None)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(bf.match(des1, des2), key=lambda m: m.distance)
vis = cv2.drawMatches(img1, kp1, img2, kp2, matches[:50], None)
```

Application: panorama stitching and image alignment

Given matched points, estimate a **homography** H, a 3x3 matrix that maps one image plane to another, robustly with **RANSAC**, which repeatedly fits a model to random minimal subsets and keeps the one explained by the most inliers.

```
src = np.float32([kp1[m.queryIdx].pt for m in matches]).reshape(-1, 1, 2)
dst = np.float32([kp2[m.trainIdx].pt for m in matches]).reshape(-1, 1, 2)
H, inliers = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
warped = cv2.warpPerspective(img1, H, (img2.shape[1] * 2, img2.shape[0]))
```

10.9 Camera geometry and calibration

The **pinhole camera model** maps a 3D point (X, Y, Z) to pixel coordinates via the intrinsic matrix K (focal lengths fx, fy, principal point cx, cy) and the extrinsic pose (rotation R, translation t): `s [u, v, 1]^T = K [R | t] [X, Y, Z, 1]^T`. Real lenses add **distortion**. **Calibration** with a printed chessboard estimates K and distortion coefficients (`cv2.calibrateCamera`), after which you can undistort images (`cv2.undistort`). Further topics: **stereo vision** (depth from two cameras via disparity: `Z = f B / d`), **epipolar geometry** (fundamental and essential matrices), **structure from motion**, and **SLAM**, which are the geometric foundation of 3D vision, robotics and augmented reality.

10.10 Video and real-time processing

```
cap = cv2.VideoCapture(0)                  # webcam; or a video file path
while True:
    ok, frame = cap.read()
    if not ok: break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 100, 200)
    cv2.imshow("edges", edges)
    if cv2.waitKey(1) & 0xFF == ord("q"): break
cap.release(); cv2.destroyAllWindows()
```

Classical video techniques: **background subtraction** (`cv2.createBackgroundSubtractorMOG2()`) to detect moving objects, **optical flow** (Lucas-Kanade, Farneback) to estimate motion between frames, and **object tracking** (Kalman filters, `cv2.TrackerCSRT`).

10.11 Classical recognition: HOG + SVM

The **Histogram of Oriented Gradients** descriptor divides the image into cells and records histograms of gradient directions. HOG with a linear SVM was the leading pedestrian detector before deep learning (`cv2.HOGDescriptor`). Understanding it shows what CNNs learned to automate.

10.12 Project: document scanner

1. Load a photo of a paper on a desk; convert to grayscale; blur.
2. Detect edges with Canny; dilate slightly.
3. Find external contours; pick the largest contour approximated by 4 corners.
4. Order the corners, compute `cv2.getPerspectiveTransform`, and warp to a flat top-down view.
5. Apply adaptive threshold for a clean black-and-white scan.

```
def order_points(pts):
    s = pts.sum(axis=1); d = np.diff(pts, axis=1).ravel()
    return np.float32([pts[s.argmin()], pts[d.argmin()], pts[s.argmax()], pts[d.argmax()]])

def four_point_warp(img, pts):
    tl, tr, br, bl = order_points(pts)
    W = int(max(np.linalg.norm(br - bl), np.linalg.norm(tr - tl)))
    H = int(max(np.linalg.norm(tr - br), np.linalg.norm(tl - bl)))
    dst = np.float32([[0, 0], [W - 1, 0], [W - 1, H - 1], [0, H - 1]])
    M = cv2.getPerspectiveTransform(np.float32([tl, tr, br, bl]), dst)
    return cv2.warpPerspective(img, M, (W, H))
```

EXERCISE - Chapter 10 exercises

- Count objects (coins, cells, grains of rice) with thresholding, morphology and contours.
- Detect lane lines on a road image using Canny, a region mask and the Hough transform (`cv2.HoughLinesP`).
- Build a real-time colour tracker in HSV that follows a coloured ball with your webcam.
- Stitch two overlapping photographs into a panorama using ORB, RANSAC and a homography.
- Calibrate a camera with a chessboard and measure the reduction in distortion.

CHAPTER 11

Convolutional Neural Networks

The convolutional neural network (CNN) is the architecture that launched the deep learning revolution in vision. This chapter builds your understanding from the convolution operation up to a complete, trained CNN, with every layer type explained.

11.1 Why convolutions?

Images have two properties that fully connected networks ignore:

- **Locality:** nearby pixels are strongly related; an edge or texture is a local pattern.
- **Translation equivariance:** a cat is a cat whether it appears in the left or right of the picture.

A convolutional layer exploits both with **local receptive fields** and **weight sharing**: the same small filter is applied at every position. A 3x3 filter has only 9 weights per input channel, regardless of image size, versus millions for a dense layer. Fewer parameters means less overfitting and better generalisation.

11.2 The convolution layer in detail

An input of shape (C_in, H, W) is convolved with C_out filters, each of shape (C_in, K, K). Each filter slides across the image; at each location it computes a dot product over the K x K x C_in patch, producing one number. The collection of outputs for one filter is a **feature map** (activation map). Output shape: (C_out, H_out, W_out).

Hyperparameter	Meaning	Typical values
Kernel size K	Size of the filter window	3 (most common), 5, 7, 1
Stride S	Step between filter positions	1 or 2
Padding P	Zeros added around the border	K//2 to preserve size ("same")
Dilation	Gaps between kernel elements; enlarges the receptive field	1, 2, 4
Groups	Splits channels into independent groups	1, or C_in for depthwise

The output size formula is `H_out = floor((H + 2P - K) / S) + 1`. For example, a 32x32 input with K=3, S=1, P=1 stays 32x32; with S=2 it becomes 16x16. The number of parameters is `C_out * (C_in * K * K + 1)`.

```
import torch, torch.nn as nn
conv = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride=1, padding=1)
x = torch.randn(8, 3, 32, 32)             # batch of 8 RGB images
print(conv(x).shape)                      # torch.Size([8, 16, 32, 32])
print(sum(p.numel() for p in conv.parameters()))   # 16*(3*3*3)+16 = 448
```

What do filters learn?

Early layers learn edges, colours and simple textures; middle layers combine them into corners, patterns and object parts; deep layers represent whole objects. This **hierarchy of features** is why depth matters. You can visualise first-layer filters of a pretrained model and see oriented edge detectors resembling the Sobel and Gabor filters of Chapter 10.

The receptive field

The **receptive field** of a unit is the region of the input that influences it. Stacking two 3x3 convolutions gives a 5x5 receptive field; three give 7x7, using fewer parameters (27 vs 49 per channel) and more non-linearity than a single 7x7 convolution. This insight shaped VGG. Strided convolutions, pooling, and dilation grow the receptive field faster.

1x1 convolutions

A 1x1 convolution mixes channels at each pixel without looking at neighbours. It is used to change the number of channels cheaply (bottlenecks), add non-linearity, and implement per-pixel classifiers in segmentation.

Depthwise separable convolutions

A standard convolution mixes space and channels at once. A **depthwise separable** convolution does a per-channel spatial convolution (`groups=C_in`) followed by a 1x1 pointwise convolution. It needs roughly 8 to 9 times fewer computations for 3x3 kernels, and powers efficient networks like MobileNet and EfficientNet.

```
depthwise = nn.Conv2d(64, 64, 3, padding=1, groups=64)
pointwise = nn.Conv2d(64, 128, 1)
```

11.3 Pooling

Pooling downsamples feature maps, reducing computation and providing small translation invariance.

```
nn.MaxPool2d(kernel_size=2, stride=2)      # keeps the strongest activation in each 2x2 window
nn.AvgPool2d(2)
nn.AdaptiveAvgPool2d(1)                    # global average pooling: (N,C,H,W) -> (N,C,1,1)
```

Global average pooling replaces large fully connected layers at the end of modern CNNs: it averages each feature map to a single number, giving a compact feature vector for the classifier and allowing any input size.

11.4 Building a complete CNN

A typical pattern is repeated blocks of **Conv, BatchNorm, ReLU** with occasional downsampling, followed by global pooling and a linear classifier.

```
def block(cin, cout):
    return nn.Sequential(
        nn.Conv2d(cin, cout, 3, padding=1, bias=False),
        nn.BatchNorm2d(cout),
        nn.ReLU(inplace=True))

class SmallCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            block(3, 32), block(32, 32), nn.MaxPool2d(2),     # 32 -> 16
            block(32, 64), block(64, 64), nn.MaxPool2d(2),    # 16 -> 8
```

```
            block(64, 128), block(128, 128), nn.MaxPool2d(2), # 8 -> 4
            nn.AdaptiveAvgPool2d(1))
        self.classifier = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x.flatten(1))
```

(`bias=False` before BatchNorm because the normalisation makes the bias redundant.)

Training on CIFAR-10

CIFAR-10 contains 60,000 colour images (32x32) in 10 classes.

```
from torchvision import datasets, transforms as T
from torch.utils.data import DataLoader

mean, std = (0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616)
train_tfm = T.Compose([T.RandomCrop(32, padding=4), T.RandomHorizontalFlip(),
                       T.ToTensor(), T.Normalize(mean, std)])
test_tfm = T.Compose([T.ToTensor(), T.Normalize(mean, std)])
train_ds = datasets.CIFAR10("data", True, train_tfm, download=True)
test_ds = datasets.CIFAR10("data", False, test_tfm, download=True)
train_loader = DataLoader(train_ds, 128, shuffle=True, num_workers=2)
test_loader = DataLoader(test_ds, 256, num_workers=2)

model = SmallCNN().to(device)
opt = torch.optim.SGD(model.parameters(), lr=0.05, momentum=0.9, weight_decay=5e-4)
sched = torch.optim.lr_scheduler.OneCycleLR(opt, 0.1, total_steps=30 * len(train_loader))
# use the train_one_epoch / evaluate functions from Chapter 8, calling sched.step() per batch
# expected: about 90 percent test accuracy after 30 epochs
```

This small network with augmentation and a schedule reaches roughly 90 percent; the MLP of Chapter 8 would manage about 55 percent. That gap is the power of convolutional structure.

11.5 Inspecting what a CNN learns

```
# visualise first-layer filters
w = model.features[0][0].weight.detach().cpu()     # (32, 3, 3, 3)
w = (w - w.min()) / (w.max() - w.min())
fig, axes = plt.subplots(4, 8, figsize=(8, 4))
for ax, f in zip(axes.ravel(), w):
    ax.imshow(f.permute(1, 2, 0)); ax.axis("off")

# feature maps via forward hooks
maps = {}
def hook(m, i, o): maps["f"] = o.detach()
model.features[0].register_forward_hook(hook)
model(images[:1].to(device))
```

Explainability: Grad-CAM

Grad-CAM highlights the regions that most influenced a class decision: take the gradients of the class score with respect to the last convolutional feature maps, average them spatially to get channel weights, form a weighted sum of the maps, apply ReLU, and upsample over the image. It reveals whether a model looks at the object or at spurious background cues (the "Clever Hans" problem: a classifier detecting horses by a copyright watermark). Libraries: `pytorch-grad-cam`, `captum`.

11.6 Convolution variants you must know

- **Transposed convolution** ("deconvolution"): learnable upsampling used in decoders and segmentation. `nn.ConvTranspose2d`.

- **Dilated (atrous) convolution:** wider receptive field without downsampling; used in DeepLab.
- **Deformable convolution:** learns sampling offsets, adapting to object shape.
- **3D convolution:** adds time or depth; for video and volumetric medical scans.
- **Grouped convolution:** used in ResNeXt.
- **Upsampling:** `nn.Upsample` (bilinear or nearest) plus a convolution is often cleaner than transposed convolution.

11.7 Dimensions cheat-sheet

Layer	Input	Output
Conv2d(Cin, Cout, K, S, P)	(N, Cin, H, W)	(N, Cout, (H+2P-K)/S+1, ...)
MaxPool2d(2)	(N, C, H, W)	(N, C, H/2, W/2)
AdaptiveAvgPool2d(1)	(N, C, H, W)	(N, C, 1, 1)
Flatten	(N, C, 1, 1)	(N, C)
Linear(C, K)	(N, C)	(N, K)

EXERCISE - Chapter 11 exercises

• Implement 2D convolution with a given stride and padding in NumPy and compare it with `nn.Conv2d` on random data.

• Compute by hand the output shape and parameter count for each layer of SmallCNN; verify with `torchsummary` or by printing shapes.

• Train SmallCNN on CIFAR-10 and plot the loss curves; then remove BatchNorm and compare.

• Visualise first-layer filters and feature maps for one image; describe what they detect.

• Implement Grad-CAM (about 25 lines using hooks) and overlay it on test images, including misclassified ones.

CHAPTER 12

Classic Architectures and Transfer Learning

A century of progress is compressed into a few architectures. Studying them teaches design principles that still guide modern models, and gives you the pretrained backbones used in nearly every real project.

12.1 The ImageNet era

ImageNet (about 1.28 million training images, 1,000 classes) and its annual challenge (ILSVRC) drove rapid progress. Top-5 error fell from 26 percent in 2011 to below human level within five years.

Model (year)	Key ideas	Why it matters
LeNet-5 (1998)	Conv, pooling, small net	First successful CNN (digits)
AlexNet (2012)	ReLU, dropout, GPU training, data augmentation	Ignited deep learning
VGG (2014)	Stacks of 3x3 convs, depth 16 to 19	Simplicity; still used as feature extractor
GoogLeNet / Inception (2014)	Parallel multi-scale branches, 1x1 bottlenecks	Efficiency
ResNet (2015)	Residual (skip) connections, up to 152 layers	Solved depth degradation; ubiquitous backbone
DenseNet (2017)	Each layer connects to all later layers	Feature reuse
MobileNet (2017+)	Depthwise separable convolutions	Efficient mobile models
EfficientNet (2019)	Compound scaling of depth, width, resolution	Best accuracy per FLOP
ConvNeXt (2022)	Modernised ResNet with transformer-era design	Shows CNNs remain competitive

12.2 ResNet: the idea you must understand

Naively stacking more layers made networks harder to train: a 56-layer plain network had **higher training error** than a 20-layer one. This is the **degradation problem**, not overfitting. ResNet's solution: let each block learn a **residual** `F(x)` and output `F(x) + x`. If the optimal transformation is close to identity, the block only needs to push `F(x)` toward zero, which is easy. The skip connection also provides a direct path for gradients, curing vanishing gradients.

```
class BasicBlock(nn.Module):
    def __init__(self, cin, cout, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(cin, cout, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(cout)
```

```
        self.conv2 = nn.Conv2d(cout, cout, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(cout)
        self.shortcut = nn.Identity()
        if stride != 1 or cin != cout:           # match shape with a 1x1 conv
            self.shortcut = nn.Sequential(
                nn.Conv2d(cin, cout, 1, stride, bias=False), nn.BatchNorm2d(cout))

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return F.relu(out + self.shortcut(x))    # the skip connection
```

ResNet-50 and deeper use **bottleneck blocks** (1x1 reduce, 3x3, 1x1 expand) to save computation. Stack four stages of blocks, doubling channels and halving resolution at each stage, then global average pooling and a linear layer: this "stem, stages, head" pattern is the template of almost every backbone.

12.3 Transfer learning

Training a deep model from scratch needs huge data and compute. **Transfer learning** reuses a network pretrained on a large dataset (such as ImageNet) and adapts it to your task. Early layers capture generic features (edges, textures) useful everywhere; only the later layers are task-specific. With transfer learning, a few hundred images per class can reach excellent accuracy.

Strategy 1: feature extraction

Freeze the pretrained backbone and train only a new classification head. Fast, safe with tiny datasets.

Strategy 2: fine-tuning

Unfreeze some or all layers and train with a **small learning rate** (about 10 times smaller than the head's) so pretrained knowledge is refined rather than destroyed. Recommended: train the head first for a few epochs, then fine-tune the whole network.

Your dataset	Similar to ImageNet	Different from ImageNet
Small	Feature extraction (train only head)	Fine-tune earlier layers carefully, strong augmentation
Large	Fine-tune all layers	Fine-tune all or train from scratch; consider domain-specific pretraining

Complete example: fine-tuning ResNet-18 on your own dataset

```
import torch, torch.nn as nn
from torchvision import models, datasets, transforms as T
from torch.utils.data import DataLoader, random_split

device = "cuda" if torch.cuda.is_available() else "cpu"
norm = T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
train_tfm = T.Compose([T.RandomResizedCrop(224), T.RandomHorizontalFlip(), T.ToTensor(), norm])
val_tfm = T.Compose([T.Resize(256), T.CenterCrop(224), T.ToTensor(), norm])

full = datasets.ImageFolder("dataset")            # dataset/classA/*.jpg, dataset/classB/*.jpg
n_val = int(0.2 * len(full))
train_idx, val_idx = random_split(range(len(full)), [len(full) - n_val, n_val],
                                  generator=torch.Generator().manual_seed(0))
train_ds = torch.utils.data.Subset(datasets.ImageFolder("dataset", train_tfm), train_idx.indices)
val_ds = torch.utils.data.Subset(datasets.ImageFolder("dataset", val_tfm), val_idx.indices)
train_loader = DataLoader(train_ds, 32, shuffle=True, num_workers=2)
val_loader = DataLoader(val_ds, 64, num_workers=2)

model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
for p in model.parameters():
    p.requires_grad = False                       # freeze the backbone
model.fc = nn.Linear(model.fc.in_features, len(full.classes))   # new trainable head
model = model.to(device)

criterion = nn.CrossEntropyLoss()
opt = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)
# Phase 1: train the head for about 5 epochs (use the loops from Chapter 8)

for p in model.parameters():
    p.requires_grad = True                        # Phase 2: unfreeze everything
opt = torch.optim.AdamW([
    {"params": [p for n, p in model.named_parameters() if not n.startswith("fc")], "lr": 1e-4},
    {"params": model.fc.parameters(), "lr": 1e-3}])
# train another 10 to 20 epochs with a cosine schedule
```

Notes: the split above is by image; if your images come from sequences or patients, split by group. When the backbone is frozen, BatchNorm layers still update their running statistics unless you put them in eval mode; either keep them in `eval()` or fine-tune everything.

Using timm: hundreds of pretrained models

```
import timm
model = timm.create_model("convnext_tiny", pretrained=True, num_classes=5)
print(timm.list_models("efficientnet*", pretrained=True)[:5])
```

The `timm` library offers ResNets, EfficientNets, ConvNeXts, ViTs, Swin and many more, with consistent APIs and published ImageNet accuracy.

12.4 Choosing a backbone

Need	Good choices
Quick baseline	ResNet-18/50, EfficientNet-B0
Mobile / edge deployment	MobileNetV3, EfficientNet-Lite, ShuffleNet
Best accuracy, moderate compute	ConvNeXt, EfficientNetV2, Swin Transformer
Maximum accuracy, large data	ViT-L, EVA, DINOv2 backbones
Small data	Self-supervised pretrained (DINOv2, CLIP) with a linear probe

12.5 Model compression and efficiency

- **Knowledge distillation:** train a small student to mimic a large teacher's soft outputs (loss includes `KL(teacher || student)` at temperature T).
- **Pruning:** remove small or unimportant weights or channels.
- **Quantisation:** use 8-bit or 4-bit integers instead of 32-bit floats; typically 2 to 4 times faster with small accuracy loss. PyTorch offers dynamic, static and quantisation-aware training.
- **Neural architecture search** automated EfficientNet-style designs.
- **Measuring cost:** parameters, **FLOPs**, memory and, most importantly, real latency on the target hardware.

12.6 Evaluation for classification

Beyond top-1 accuracy: **top-5 accuracy**, per-class precision/recall, confusion matrix, **calibration** (do predicted probabilities match actual frequencies? measure with Expected Calibration Error; fix with temperature scaling), and **robustness** (performance under blur, noise, lighting change: see ImageNet-C). Always inspect the failure cases visually.

EXERCISE - Chapter 12 exercises

• Fine-tune ResNet-18 on a small dataset you collect yourself (for example, three kinds of local fruit, 100 photos each); report accuracy with feature extraction versus full fine-tuning.

• Implement ResNet-18 from scratch using the BasicBlock above; check that the parameter count is about 11.7 million; train it on CIFAR-10.

• Compare ResNet-18, EfficientNet-B0 and ConvNeXt-Tiny from timm on the same task in terms of accuracy, parameters and inference speed.

• Apply knowledge distillation from ResNet-50 to a small CNN on CIFAR-10.

• Quantise a trained model to int8 and measure speed and accuracy change.

CHAPTER 13

Object Detection

Classification says *what* is in an image. **Object detection** says *what and where*: it outputs a set of bounding boxes, each with a class label and a confidence score. Detection powers self-driving perception, retail analytics, surveillance, medical screening and robotics.

13.1 The problem and the vocabulary

A **bounding box** is represented as `(x1, y1, x2, y2)` (corners) or `(cx, cy, w, h)` (centre and size) or `(x, y, w, h)` (top-left plus size). Conventions vary between datasets and libraries, and mixing them up is a very common bug. Coordinates may be absolute pixels or normalised to [0, 1].

Intersection over Union (IoU)

IoU measures how much two boxes overlap: `IoU = area(A and B) / area(A or B)`. It ranges from 0 (disjoint) to 1 (identical). A prediction is typically a **true positive** if IoU with a ground-truth box of the same class exceeds a threshold (often 0.5).

```
def iou(a, b):
    ix1, iy1 = max(a[0], b[0]), max(a[1], b[1])
    ix2, iy2 = min(a[2], b[2]), min(a[3], b[3])
    inter = max(0, ix2 - ix1) * max(0, iy2 - iy1)
    area_a = (a[2] - a[0]) * (a[3] - a[1])
    area_b = (b[2] - b[0]) * (b[3] - b[1])
    return inter / (area_a + area_b - inter + 1e-9)

print(iou((0, 0, 10, 10), (5, 5, 15, 15)))    # 25 / 175 = 0.1428
```

Non-maximum suppression (NMS)

Detectors produce many overlapping boxes for the same object. NMS keeps the highest-scoring box and discards others of the same class whose IoU with it exceeds a threshold, repeating for the remaining boxes.

```
def nms(boxes, scores, thr=0.5):
    order = np.argsort(scores)[::-1]
    keep = []
    while len(order):
        i = order[0]; keep.append(i)
        rest = order[1:]
        order = np.array([j for j in rest if iou(boxes[i], boxes[j]) < thr], dtype=int)
    return keep
# torchvision.ops.nms(boxes, scores, iou_threshold) does this efficiently
```

Variants: Soft-NMS (decays scores instead of removing), and NMS-free end-to-end detectors (DETR family).

13.2 Evaluation: precision, recall, AP and mAP

For each class, rank all predictions by confidence; walking down the list, mark each prediction as TP or FP (by IoU matching with unmatched ground truth), and compute the precision-recall curve. **Average Precision (AP)** is the area under this curve. **mAP** is the mean of AP over classes.

Metric	Meaning
AP50 (mAP@0.5)	AP with IoU threshold 0.5 (PASCAL VOC style)
AP75	Stricter localisation, threshold 0.75
mAP@[.5:.95] (COCO mAP)	Average of AP over IoU thresholds 0.5, 0.55, ..., 0.95; the main COCO metric
AP small / medium / large	Performance by object size

Use `pycocotools` or `torchmetrics.detection.MeanAveragePrecision` rather than writing your own.

13.3 Datasets and annotation formats

- **PASCAL VOC:** XML per image, boxes as `xmin, ymin, xmax, ymax`.
- **COCO:** one JSON file with `images`, `annotations` (boxes as `[x, y, width, height]`, segmentation, category_id), `categories`. The de facto standard (80 classes, 330K images).
- **YOLO format:** one text file per image, each line `class cx cy w h` normalised to [0, 1].
- Others: Open Images, Objects365, LVIS (long tail), KITTI and nuScenes (driving).

```
# convert a pixel box (x1,y1,x2,y2) to YOLO format
def to_yolo(box, W, H):
    x1, y1, x2, y2 = box
    return ((x1 + x2) / 2 / W, (y1 + y2) / 2 / H, (x2 - x1) / W, (y2 - y1) / H)
```

Annotation tools: **CVAT**, **Label Studio**, **Roboflow**, **LabelImg**.

13.4 Two families of detectors

Two-stage detectors: R-CNN family

1. **R-CNN (2014):** propose ~2000 regions (Selective Search), run a CNN on each. Accurate, extremely slow.
2. **Fast R-CNN:** run the CNN once, then pool features for each region (**RoI pooling**).
3. **Faster R-CNN (2015):** adds a **Region Proposal Network (RPN)** that predicts objectness and box offsets for **anchor boxes** (predefined boxes of several scales and aspect ratios at every feature-map location). Stage two classifies each proposal and refines its box.
4. **Mask R-CNN:** adds a mask branch for instance segmentation (Chapter 14) and RoIAlign for precise alignment.
5. **Feature Pyramid Network (FPN):** builds multi-scale feature maps so small and large objects are both detected well; used by nearly all modern detectors.

Two-stage detectors are accurate and strong with small objects but slower.

One-stage detectors: SSD, RetinaNet, YOLO

They predict class and box directly from dense grid positions in one pass, favouring speed.

- **YOLO (You Only Look Once):** divides the image into a grid; each cell predicts boxes, objectness and class probabilities. The family evolved rapidly (YOLOv3, v5, v7, v8, v9, v10, v11 and beyond), adding better backbones, FPN/PAN necks, anchor-free heads, advanced augmentation (mosaic) and improved losses.

- **SSD:** multi-scale feature maps with default boxes.
- **RetinaNet:** introduced the **focal loss** `FL = -alpha (1 - p_t)^gamma log(p_t)`, which down-weights the overwhelming number of easy background examples and closed the accuracy gap with two-stage models.
- **FCOS / CenterNet:** anchor-free detectors predicting centre points or per-pixel distances to box edges.

Transformer-based detectors

DETR (2020) treats detection as set prediction: a CNN backbone feeds a transformer encoder-decoder that outputs a fixed set of object queries, trained with **Hungarian matching** between predictions and ground truth. No anchors and no NMS. Successors (Deformable DETR, DINO, RT-DETR) fixed slow convergence and reached state-of-the-art accuracy with real-time speed. **Open-vocabulary detectors** (OWL-ViT, Grounding DINO, YOLO-World) detect classes described by text prompts without retraining.

13.5 Detection losses

A detector minimises a sum of: a **classification loss** (cross-entropy or focal), a **box regression loss** (smooth L1, or IoU-based losses: GIoU, DIoU, CIoU, which optimise overlap directly), and an **objectness loss**. Anchor assignment (which anchors count as positives) strongly affects results; modern methods use dynamic assignment (SimOTA, TAL).

13.6 Hands-on: pretrained detection with torchvision

```
import torch, torchvision
from torchvision.models.detection import fasterrcnn_resnet50_fpn_v2, FasterRCNN_ResNet50_FPN_V2_Weights
from torchvision.io import read_image
from torchvision.utils import draw_bounding_boxes

weights = FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT
model = fasterrcnn_resnet50_fpn_v2(weights=weights, box_score_thresh=0.5).eval()
preprocess = weights.transforms()
img = read_image("street.jpg")
with torch.no_grad():
    out = model([preprocess(img)])[0]          # dict: boxes, labels, scores
names = [weights.meta["categories"][i] for i in out["labels"]]
vis = draw_bounding_boxes(img, out["boxes"], names, colors="lime", width=3)
plt.imshow(vis.permute(1, 2, 0)); plt.axis("off")
```

13.7 Hands-on: training YOLO on your own data

With the Ultralytics library (`pip install ultralytics`), training a modern YOLO takes minutes to set up.

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")                    # small pretrained model
model.train(data="data.yaml", epochs=100, imgsz=640, batch=16)
metrics = model.val()                         # mAP50, mAP50-95, per-class results
results = model("test.jpg", conf=0.25)        # inference
results[0].show()
model.export(format="onnx")                   # deployment format
```

`data.yaml` describes the dataset:

```
path: datasets/mydata
train: images/train
val: images/val
```

```
names:
  0: helmet
  1: vest
```

Folder layout: `images/train/*.jpg` and `labels/train/*.txt` (YOLO format). Steps for a real project: (1) collect 200 to 2,000 diverse images, (2) annotate boxes carefully and consistently, (3) split by source not by frame, (4) train from a pretrained checkpoint, (5) examine the confusion matrix and failure cases, (6) add data for the failure modes, (7) tune the confidence and NMS thresholds on validation data.

13.8 Fine-tuning torchvision Faster R-CNN on a custom dataset

```
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

model = fasterrcnn_resnet50_fpn_v2(weights=weights)
in_f = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor = FastRCNNPredictor(in_f, num_classes=3)   # 2 classes + background

# the Dataset must return (image_tensor, target) where
# target = {"boxes": FloatTensor[N,4] (x1,y1,x2,y2), "labels": Int64Tensor[N]}
def collate(batch): return tuple(zip(*batch))
loader = DataLoader(ds, batch_size=4, shuffle=True, collate_fn=collate)

params = [p for p in model.parameters() if p.requires_grad]
opt = torch.optim.SGD(params, lr=0.005, momentum=0.9, weight_decay=5e-4)
model.train()
for images, targets in loader:
    images = [i.to(device) for i in images]
    targets = [{k: v.to(device) for k, v in t.items()} for t in targets]
    loss_dict = model(images, targets)         # in train mode the model returns losses
    loss = sum(loss_dict.values())
    opt.zero_grad(); loss.backward(); opt.step()
```

The custom `collate_fn` is needed because every image has a different number of boxes.

13.9 Practical issues

- **Small objects:** use higher input resolution, FPN, tiling (SAHI), and appropriate anchor sizes.
- **Class imbalance and background:** focal loss, hard negative mining, balanced sampling.
- **Crowded scenes and occlusion:** tune NMS; consider DETR-style models.
- **Domain shift:** a model trained in daylight fails at night; collect data across conditions.
- **Speed:** measure end to end (preprocessing, model, NMS). Export to ONNX or TensorRT; use FP16 or INT8.
- **Thresholds:** choose the confidence threshold from the precision-recall curve for your application.

13.10 Multi-object tracking

Detection works per frame; **tracking** links detections across frames to maintain identities. The standard approach is **tracking-by-detection**: a Kalman filter predicts each track's next box, detections are assigned to tracks by the Hungarian algorithm using IoU (**SORT**) and appearance embeddings (**DeepSORT**, **ByteTrack**, **BoT-SORT**). Metrics: **MOTA**, **IDF1**, **HOTA**. Ultralytics provides `model.track(source="video.mp4")`.

EXERCISE - Chapter 13 exercises

• Implement IoU, NMS and AP for one class from scratch and test them against `torchvision.ops` and `pycocotools`.

• Run a pretrained detector on 20 of your own photographs and categorise the errors (missed, wrong class, duplicate, poorly localised).

• Train YOLO on a small custom dataset (for example, bottles on a desk, annotated yourself in CVAT) and report mAP50 and mAP50-95.

• Build a people-counting application with YOLO and a tracker that counts crossings of a line in a video.

• Compare a one-stage and a two-stage model on speed versus accuracy on the same data.

CHAPTER 14

Image Segmentation

Segmentation gives the most detailed understanding of an image: a label for **every pixel**. It is essential for medical imaging, autonomous driving, satellite analysis, photo editing and robotics.

14.1 Three kinds of segmentation

Task	Output	Example
Semantic segmentation	Class label for every pixel; no separation of individual objects	All road pixels, all car pixels
Instance segmentation	A separate mask for each object instance	Car 1, car 2, car 3 each with their own mask
Panoptic segmentation	Semantic labels for "stuff" (sky, road) plus instances for "things" (cars, people)	Complete scene parsing

14.2 Data and metrics

A segmentation mask is an image of the same height and width whose pixel values are class indices (0 = background, 1 = class 1, ...). Datasets: PASCAL VOC, COCO (instance), Cityscapes (urban driving), ADE20K (scene parsing), plus numerous medical datasets. When resizing masks use **nearest-neighbour** interpolation so class IDs are not blended.

Metrics

- **Pixel accuracy:** fraction of correctly labelled pixels; misleading when background dominates.
- **IoU (Jaccard) per class:** `TP / (TP + FP + FN)`; **mean IoU (mIoU)** averages over classes: the standard semantic metric.
- **Dice coefficient (F1):** `2 TP / (2 TP + FP + FN)`; standard in medical imaging.
- **Panoptic Quality (PQ)** for panoptic segmentation; **mask AP** for instance segmentation.

```
def dice_iou(pred, target, eps=1e-7):          # binary masks as 0/1 tensors
    inter = (pred * target).sum()
    dice = (2 * inter + eps) / (pred.sum() + target.sum() + eps)
    iou = (inter + eps) / (pred.sum() + target.sum() - inter + eps)
    return dice.item(), iou.item()
```

14.3 Loss functions

- **Pixel-wise cross-entropy:** the default; `nn.CrossEntropyLoss` accepts logits (N, C, H, W) and targets (N, H, W).
- **Weighted cross-entropy:** compensates for rare classes.
- **Dice loss:** `1 - Dice`; directly optimises overlap; robust to imbalance. Often combined: `loss = CE + Dice`.

- **Focal loss, Tversky loss, Lovasz-softmax:** other imbalance-aware and IoU-aware options.

```
class DiceLoss(nn.Module):
    def forward(self, logits, target):          # logits (N,1,H,W), target (N,1,H,W) in {0,1}
        p = torch.sigmoid(logits)
        inter = (p * target).sum((2, 3))
        denom = p.sum((2, 3)) + target.sum((2, 3))
        return (1 - (2 * inter + 1) / (denom + 1)).mean()
```

14.4 Fully convolutional networks

FCN (2015) replaced the fully connected layers of a classifier with convolutions so the network accepts any input size and outputs a spatial map of class scores, then upsampled them to input resolution. Skip connections from earlier layers restore fine detail lost by downsampling. This idea underlies all that follows.

14.5 The encoder-decoder and U-Net

U-Net (2015), designed for biomedical segmentation with very little data, is the most influential segmentation architecture. An **encoder** (contracting path) repeatedly convolves and downsamples to capture context; a **decoder** (expanding path) upsamples and recovers detail; **skip connections** concatenate encoder features with decoder features at matching resolutions, combining "what" (deep semantics) with "where" (fine localisation).

```
class DoubleConv(nn.Sequential):
    def __init__(self, cin, cout):
        super().__init__(
            nn.Conv2d(cin, cout, 3, padding=1, bias=False), nn.BatchNorm2d(cout), nn.ReLU(True),
            nn.Conv2d(cout, cout, 3, padding=1, bias=False), nn.BatchNorm2d(cout), nn.ReLU(True))

class UNet(nn.Module):
    def __init__(self, in_ch=3, n_classes=1, base=32):
        super().__init__()
        self.d1, self.d2 = DoubleConv(in_ch, base), DoubleConv(base, base * 2)
        self.d3, self.d4 = DoubleConv(base * 2, base * 4), DoubleConv(base * 4, base * 8)
        self.pool = nn.MaxPool2d(2)
        self.up3 = nn.ConvTranspose2d(base * 8, base * 4, 2, 2)
        self.u3 = DoubleConv(base * 8, base * 4)
        self.up2 = nn.ConvTranspose2d(base * 4, base * 2, 2, 2)
        self.u2 = DoubleConv(base * 4, base * 2)
        self.up1 = nn.ConvTranspose2d(base * 2, base, 2, 2)
        self.u1 = DoubleConv(base * 2, base)
        self.head = nn.Conv2d(base, n_classes, 1)

    def forward(self, x):
        c1 = self.d1(x)
        c2 = self.d2(self.pool(c1))
        c3 = self.d3(self.pool(c2))
        c4 = self.d4(self.pool(c3))                         # bottleneck
        x = self.u3(torch.cat([self.up3(c4), c3], 1))       # skip connection by concatenation
        x = self.u2(torch.cat([self.up2(x), c2], 1))
        x = self.u1(torch.cat([self.up1(x), c1], 1))
        return self.head(x)                                 # (N, n_classes, H, W) logits

print(UNet()(torch.randn(2, 3, 128, 128)).shape)            # torch.Size([2, 1, 128, 128])
```

(Input height and width must be divisible by 8 here.) In practice, use a **pretrained encoder** via `segmentation_models_pytorch`:

```
import segmentation_models_pytorch as smp
model = smp.Unet("resnet34", encoder_weights="imagenet", in_channels=3, classes=1)
loss_fn = smp.losses.DiceLoss("binary")
```

14.6 Modern semantic segmentation architectures

- **DeepLab family:** atrous (dilated) convolutions and **ASPP** (Atrous Spatial Pyramid Pooling) capture multi-scale context; DeepLabv3+ adds a decoder.
- **PSPNet:** pyramid pooling for global context.
- **HRNet:** keeps high-resolution representations throughout.
- **SegFormer, Swin-UNet, UPerNet:** transformer-based, strong accuracy and efficiency.
- **Mask2Former:** one architecture for semantic, instance and panoptic segmentation using mask classification.

14.7 Instance segmentation

- **Mask R-CNN:** Faster R-CNN plus a small FCN predicting a binary mask for each detected box, with RoIAlign. Still a strong baseline.
- **YOLO-seg, YOLACT, SOLOv2:** faster single-stage alternatives.
- **Mask2Former / MaskFormer:** query-based, state-of-the-art.

```
from torchvision.models.detection import maskrcnn_resnet50_fpn_v2, MaskRCNN_ResNet50_FPN_V2_Weights
w = MaskRCNN_ResNet50_FPN_V2_Weights.DEFAULT
model = maskrcnn_resnet50_fpn_v2(weights=w).eval()
with torch.no_grad():
    out = model([w.transforms()(img)])[0]       # boxes, labels, scores, masks (N,1,H,W)
masks = out["masks"][out["scores"] > 0.6] > 0.5
```

14.8 Foundation models for segmentation: SAM

The **Segment Anything Model (SAM)** and **SAM 2** are promptable segmentation models trained on over a billion masks. You give a point, a box, or text-grounded prompt, and it outputs high-quality masks of nearly any object, without task-specific training. Uses: accelerating annotation (auto-label then correct), interactive tools, and as a component in pipelines (Grounding DINO plus SAM = "segment by text"). Fine-tune adapters for specialised domains such as medical scans.

14.9 Training pipeline example

```
class SegDataset(Dataset):
    def __init__(self, img_paths, mask_paths, transform):
        self.imgs, self.masks, self.t = img_paths, mask_paths, transform
    def __len__(self): return len(self.imgs)
    def __getitem__(self, i):
        img = cv2.cvtColor(cv2.imread(str(self.imgs[i])), cv2.COLOR_BGR2RGB)
        mask = cv2.imread(str(self.masks[i]), cv2.IMREAD_GRAYSCALE)
        out = self.t(image=img, mask=mask)          # Albumentations transforms both together
        return out["image"], out["mask"].long()

import albumentations as A
from albumentations.pytorch import ToTensorV2
train_t = A.Compose([A.RandomResizedCrop(size=(256, 256)), A.HorizontalFlip(),
                     A.RandomBrightnessContrast(), A.Normalize(), ToTensorV2()])
```

Spatial augmentations (flip, crop, rotate) **must be applied identically to image and mask**; colour augmentations apply to the image only (Albumentations handles this). Train with `CE + Dice`, AdamW at 1e-3 for the decoder, and 1e-4 for the pretrained encoder; evaluate with mIoU or Dice; and visualise predictions overlaid on images every few epochs.

14.10 Special topics

- **Medical imaging:** 3D volumes (nnU-Net is the strongest self-configuring baseline), DICOM and NIfTI formats, class imbalance, Dice loss, patient-level splits, regulatory and ethical considerations.
- **Remote sensing:** very large images processed in tiles, multispectral bands, change detection.
- **Real-time segmentation:** BiSeNet, PIDNet, efficient encoders.
- **Post-processing:** conditional random fields, morphological cleaning, connected-component filtering.

EXERCISE - Chapter 14 exercises

• Implement Dice and IoU metrics; verify them on hand-drawn masks.

• Train the U-Net above on a binary segmentation dataset (for example, Oxford-IIIT Pet trimaps or a Kaggle medical dataset); report Dice.

• Replace the encoder with a pretrained ResNet-34 (smp) and quantify the improvement.

• Run Mask R-CNN and SAM on the same images; compare mask quality and speed.

• Build a "background remover" combining a segmentation model with OpenCV compositing.

CHAPTER 15

Transformers, Self-Supervised Learning and Foundation Models

Since 2020, transformers have reshaped computer vision. Today's strongest systems (DINOv2, CLIP, SAM, vision-language models) are all built on them. This chapter explains attention from first principles, the Vision Transformer, and the foundation-model paradigm.

15.1 Attention

Convolutions look at local neighbourhoods. **Attention** lets every element look at every other element and decide what is relevant. Given queries Q, keys K and values V (all produced by linear projections of the input), **scaled dot-product attention** is:

```
Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k)) V
```

Each query computes similarity scores with all keys; softmax turns them into weights; the output is a weighted average of the values. Dividing by `sqrt(d_k)` keeps scores in a range where softmax has useful gradients.

```
import torch, torch.nn as nn, torch.nn.functional as F, math

def attention(Q, K, V):
    scores = Q @ K.transpose(-2, -1) / math.sqrt(Q.size(-1))   # (..., T, T)
    return F.softmax(scores, dim=-1) @ V

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, dim, heads=8):
        super().__init__()
        self.h, self.dk = heads, dim // heads
        self.qkv = nn.Linear(dim, dim * 3)
        self.out = nn.Linear(dim, dim)
    def forward(self, x):                                  # x: (N, T, dim)
        N, T, D = x.shape
        q, k, v = self.qkv(x).reshape(N, T, 3, self.h, self.dk).permute(2, 0, 3, 1, 4)
        y = attention(q, k, v)                             # (N, heads, T, dk)
        return self.out(y.transpose(1, 2).reshape(N, T, D))
```

Multi-head attention runs several attention operations in parallel in different subspaces, letting the model attend to different kinds of relationships at once. The cost is quadratic in sequence length T, which is why efficiency research (windowed attention, linear attention, FlashAttention) matters for high-resolution images.

15.2 The transformer block

A transformer block consists of: LayerNorm, multi-head self-attention, residual connection; LayerNorm, an MLP (two linear layers with GELU), residual connection.

```
class TransformerBlock(nn.Module):
    def __init__(self, dim, heads, mlp_ratio=4, drop=0.0):
        super().__init__()
        self.n1, self.attn = nn.LayerNorm(dim), MultiHeadSelfAttention(dim, heads)
        self.n2 = nn.LayerNorm(dim)
        self.mlp = nn.Sequential(nn.Linear(dim, dim * mlp_ratio), nn.GELU(),
                                 nn.Dropout(drop), nn.Linear(dim * mlp_ratio, dim))
```

```
    def forward(self, x):
        x = x + self.attn(self.n1(x))
        return x + self.mlp(self.n2(x))
```

Attention is permutation-invariant, so **positional information** must be added to tokens (learned or sinusoidal position embeddings, or relative/rotary encodings).

15.3 The Vision Transformer (ViT)

ViT (2020) applies a standard transformer to images by treating an image as a sequence of patches:

1. Split the image into non-overlapping patches (for example 16x16): a 224x224 image gives 14 x 14 = 196 patches.
2. Flatten each patch and project it linearly to a vector of size `dim` (the **patch embedding**; implemented as a convolution with kernel = stride = patch size).
3. Prepend a learnable **[CLS] token** and add learned positional embeddings.
4. Pass through L transformer blocks.
5. Use the final [CLS] representation for classification.

```
class ViT(nn.Module):
    def __init__(self, img=224, patch=16, dim=384, depth=12, heads=6, classes=1000):
        super().__init__()
        n = (img // patch) ** 2
        self.patch = nn.Conv2d(3, dim, patch, patch)           # patch embedding
        self.cls = nn.Parameter(torch.zeros(1, 1, dim))
        self.pos = nn.Parameter(torch.zeros(1, n + 1, dim))
        self.blocks = nn.Sequential(*[TransformerBlock(dim, heads) for _ in range(depth)])
        self.norm, self.head = nn.LayerNorm(dim), nn.Linear(dim, classes)
    def forward(self, x):
        x = self.patch(x).flatten(2).transpose(1, 2)           # (N, n, dim)
        x = torch.cat([self.cls.expand(len(x), -1, -1), x], 1) + self.pos
        x = self.norm(self.blocks(x))
        return self.head(x[:, 0])
```

Key insight: ViTs have fewer built-in image assumptions (no locality or translation equivariance), so they need **much more data** or strong pretraining and augmentation (DeiT showed good ImageNet-only training with distillation and heavy augmentation). Given large-scale pretraining, they surpass CNNs and scale better.

15.4 Hierarchical and hybrid vision transformers

- **Swin Transformer:** attention within local **shifted windows**, hierarchical feature maps with patch merging, linear complexity. A general-purpose backbone for detection and segmentation.
- **PVT, MViT, CvT, CoAtNet:** pyramid or hybrid designs combining convolutions and attention.
- **ConvNeXt:** shows that a ResNet modernised with transformer-era recipes (large kernels, LayerNorm, GELU, AdamW, long schedules) matches transformers.

Lesson: the training recipe (data, augmentation, schedule, regularisation) matters as much as the architecture.

15.5 Using pretrained vision transformers

```
import timm
vit = timm.create_model("vit_base_patch16_224", pretrained=True, num_classes=10)
cfg = timm.data.resolve_data_config({}, model=vit)
tfm = timm.data.create_transform(**cfg)       # the correct preprocessing for this model
```

Fine-tuning tips: use a **lower learning rate** (1e-5 to 5e-5 with AdamW), warmup, layer-wise learning-rate decay (lower layers learn slower), strong augmentation (RandAugment, Mixup, CutMix), and weight decay 0.05.

15.6 Self-supervised learning (SSL)

Labels are expensive; unlabelled images are abundant. SSL creates a learning signal from the data itself, producing general-purpose representations that transfer superbly with few labels.

Contrastive learning

Train an encoder so that two augmented views of the same image have **similar** embeddings while views of different images are **dissimilar**. **SimCLR** uses large batches with the NT-Xent (InfoNCE) loss; **MoCo** uses a momentum encoder and a queue of negatives.

```
def info_nce(z1, z2, temperature=0.1):
    z1, z2 = F.normalize(z1, dim=1), F.normalize(z2, dim=1)
    logits = z1 @ z2.T / temperature            # (N, N) similarity matrix
    labels = torch.arange(len(z1), device=z1.device)    # positives on the diagonal
    return F.cross_entropy(logits, labels)
```

Non-contrastive and self-distillation

BYOL and **SimSiam** learn without negatives using a predictor and stop-gradient. **DINO / DINOv2** train a student to match a momentum teacher's output on different crops of the same image; DINOv2 features are extraordinary: a linear classifier or even k-NN on frozen features rivals supervised models, and the features work well for depth, segmentation and retrieval.

Masked image modelling

MAE (Masked Autoencoder) masks ~75 percent of the patches and trains a ViT to reconstruct them; the encoder sees only visible patches, making pretraining very efficient. **BEiT** predicts discrete visual tokens of masked patches. These methods mirror masked language modelling in NLP.

15.7 Vision-language models

CLIP

CLIP (2021) trains an image encoder and a text encoder on hundreds of millions of image-caption pairs with a contrastive loss so that matching pairs have similar embeddings in a shared space. Consequences:

- **Zero-shot classification:** classify by comparing the image embedding with text embeddings of prompts such as "a photo of a {class}", with no task-specific training.
- **Image-text retrieval** and semantic search.

- A powerful frozen backbone and the text-conditioning component of many generative systems.

```
# pip install open_clip_torch
import open_clip, torch
from PIL import Image
model, _, preprocess = open_clip.create_model_and_transforms("ViT-B-32", pretrained="openai")
tok = open_clip.get_tokenizer("ViT-B-32")
image = preprocess(Image.open("pet.jpg")).unsqueeze(0)
text = tok(["a photo of a cat", "a photo of a dog", "a photo of a bird"])
with torch.no_grad():
    fi, ft = model.encode_image(image), model.encode_text(text)
    fi, ft = fi / fi.norm(dim=-1, keepdim=True), ft / ft.norm(dim=-1, keepdim=True)
    print((100 * fi @ ft.T).softmax(dim=-1))        # probabilities over the prompts
```

Successors: SigLIP (sigmoid loss, better scaling), EVA-CLIP, OpenCLIP.

Large multimodal models

Modern **vision-language models (VLMs)** connect a vision encoder to a large language model (via a projection layer or cross-attention) and are trained to follow instructions about images: visual question answering, captioning, document and chart understanding, OCR, and visual reasoning. Examples include LLaVA, Qwen-VL, PaliGemma, and the multimodal assistants from major labs. As an engineer you will increasingly use VLMs as components: auto-labelling data, reading documents, or as evaluators; always validate their outputs because they can hallucinate.

15.8 The foundation-model workflow

1. Start with a strong pretrained backbone (DINOv2, CLIP, SigLIP, a SAM variant).
2. **Linear probe:** train only a linear layer on frozen features. Quick and sometimes enough.
3. **Parameter-efficient fine-tuning:** train small adapters or **LoRA** matrices, leaving the backbone mostly frozen.
4. **Full fine-tuning** with a small learning rate when you have much data.
5. For retrieval and few-shot tasks, use **embeddings plus nearest neighbour search** (FAISS).

```
# few-shot classification with frozen DINOv2 features and logistic regression
dino = torch.hub.load("facebookresearch/dinov2", "dinov2_vits14").eval().to(device)
@torch.no_grad()
def embed(loader):
    feats, ys = [], []
    for x, y in loader:
        feats.append(dino(x.to(device)).cpu()); ys.append(y)
    return torch.cat(feats).numpy(), torch.cat(ys).numpy()
Ftr, ytr = embed(train_loader); Fte, yte = embed(test_loader)
from sklearn.linear_model import LogisticRegression
print(LogisticRegression(max_iter=2000).fit(Ftr, ytr).score(Fte, yte))
```

15.9 Scaling laws and compute

Performance improves predictably with more data, parameters and compute. Practical consequences: when resources are limited, prefer a pretrained foundation model plus light adaptation over training large networks yourself; invest in data quality; and understand cost (GPU hours, memory) before you plan experiments.

EXERCISE - Chapter 15 exercises

• Implement multi-head attention from scratch and verify it against `nn.MultiheadAttention`.

• Visualise the attention map of a pretrained ViT's [CLS] token over an image.

• Fine-tune a ViT-S on a small dataset and compare it with ResNet-50 at equal training time.

• Perform CLIP zero-shot classification on your own images; experiment with prompt wording and measure the accuracy change.

• Build an image search engine: embed 5,000 images with CLIP or DINOv2, index with FAISS, and query with text and image.

CHAPTER 16

Generative Models for Vision

Generative models learn the distribution of images and can create new ones. They power image synthesis, editing, super-resolution, inpainting, data augmentation, and are an active research frontier. Even if your focus is recognition, understanding them completes your expertise.

16.1 Autoencoders

An **autoencoder** compresses an image to a small **latent code** with an encoder and reconstructs it with a decoder, minimising reconstruction error. It learns useful low-dimensional representations; **denoising autoencoders** reconstruct clean images from noisy inputs; applications include compression, anomaly detection (high reconstruction error flags unusual images), and denoising.

```
class AE(nn.Module):
    def __init__(self, latent=32):
        super().__init__()
        self.enc = nn.Sequential(nn.Flatten(), nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, latent))
        self.dec = nn.Sequential(nn.Linear(latent, 256), nn.ReLU(), nn.Linear(256, 784), nn.Sigmoid())
    def forward(self, x):
        return self.dec(self.enc(x)).view(-1, 1, 28, 28)
# loss: F.mse_loss(model(x), x)
```

16.2 Variational autoencoders (VAE)

A VAE makes the latent space **probabilistic and smooth**. The encoder outputs a mean and variance defining a Gaussian; we sample `z = mu + sigma * eps` with `eps ~ N(0, 1)` (the **reparameterisation trick**, which keeps sampling differentiable). The loss is reconstruction plus a **KL divergence** pulling the latent distribution toward a standard normal: `L = recon + KL(q(z|x) || N(0, I))`. After training, sample `z ~ N(0, I)` and decode to generate new images. VAE samples tend to be blurry, but VAEs are crucial components of latent diffusion (the autoencoder that maps between pixels and latent space).

16.3 Generative Adversarial Networks (GANs)

A **GAN** pits two networks against each other: a **generator** G maps noise z to a fake image; a **discriminator** D tries to distinguish real from fake. G learns to fool D; D learns to catch G. The minimax objective: `min_G max_D E[log D(x)] + E[log(1 - D(G(z)))]`.

```
# one training step (sketch)
real = batch.to(device); N = len(real)
z = torch.randn(N, 100, device=device)
fake = G(z)
# discriminator: real -> 1, fake -> 0
d_loss = bce(D(real), torch.ones(N, 1, device=device)) + bce(D(fake.detach()), torch.zeros(N, 1, device=device))
optD.zero_grad(); d_loss.backward(); optD.step()
# generator: make D say fake -> 1
g_loss = bce(D(fake), torch.ones(N, 1, device=device))
optG.zero_grad(); g_loss.backward(); optG.step()
```

GANs are hard to train (mode collapse, instability). Improvements: DCGAN architecture guidelines, **WGAN-GP** (Wasserstein loss with gradient penalty), spectral normalisation, **StyleGAN** (photorealistic faces with controllable style), **pix2pix** and **CycleGAN**

(image-to-image translation, paired and unpaired), **Real-ESRGAN** (super-resolution). GANs generate in a single fast pass; diffusion models have largely overtaken them in quality and diversity.

16.4 Diffusion models

Diffusion models are the foundation of modern image generation (Stable Diffusion, DALL-E, Midjourney, Imagen, Flux).

The idea

- **Forward process:** gradually add Gaussian noise to a real image over T steps until it becomes pure noise. In closed form: `x_t = sqrt(a_bar_t) x_0 + sqrt(1 - a_bar_t) eps`, where `eps ~ N(0, I)` and `a_bar_t` decreases from 1 toward 0.
- **Reverse process:** train a neural network (usually a U-Net or a transformer, "DiT") to **predict the noise** `eps` added at step t. Generation starts from random noise and repeatedly removes a little predicted noise.
- **Training loss** is simple: `L = || eps - eps_theta(x_t, t) ||^2` with a random t per sample.

```
def diffusion_train_step(model, x0, alphas_bar):
    N = len(x0)
    t = torch.randint(0, len(alphas_bar), (N,), device=x0.device)
    eps = torch.randn_like(x0)
    a = alphas_bar[t].view(N, 1, 1, 1)
    xt = a.sqrt() * x0 + (1 - a).sqrt() * eps           # noisy image
    return F.mse_loss(model(xt, t), eps)                # predict the noise
```

Making diffusion practical

- **Latent diffusion** (Stable Diffusion): run diffusion in the compact latent space of a VAE, far cheaper than pixel space.
- **Conditioning:** text embeddings (CLIP or T5) are injected through **cross-attention**; **classifier-free guidance** blends conditional and unconditional predictions to strengthen prompt adherence: `eps = eps_uncond + w (eps_cond - eps_uncond)`.
- **Fast samplers:** DDIM, DPM-Solver and distillation methods reduce 1,000 steps to 4 to 30; **flow matching / rectified flow** models learn straight transport paths and are now common.
- **Control and personalisation: ControlNet** (condition on edges, depth, pose), **LoRA** and **DreamBooth** (teach new subjects or styles with a few images), **IP-Adapter** (image prompts), inpainting and editing (InstructPix2Pix).

Using diffusion models in practice

```
# pip install diffusers transformers accelerate
from diffusers import StableDiffusionXLPipeline
import torch
pipe = StableDiffusionXLPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0", torch_dtype=torch.float16).to("cuda")
image = pipe("a watercolor painting of a lighthouse at dawn",
             num_inference_steps=30, guidance_scale=7.0).images[0]
image.save("lighthouse.png")
```

(Model names and licences change; check the Hugging Face hub for current models and their terms.)

16.5 Evaluating generative models

- **FID (Fréchet Inception Distance):** distance between feature statistics of real and generated images; lower is better.
- **Inception Score, precision/recall for generative models, CLIP score** (text-image alignment), and human preference studies.
- Always inspect samples visually and test diversity.

16.6 Applications you can build

- **Super-resolution:** SRCNN, ESRGAN, SwinIR; diffusion-based upscalers.
- **Inpainting and outpainting:** fill or extend images.
- **Denoising and restoration:** old photo repair, deblurring.
- **Synthetic data generation:** create rare cases (defects, night scenes) to train detectors, with careful validation of the domain gap.
- **Style transfer:** neural style transfer, image-to-image translation.
- **Video generation:** temporal diffusion transformers; also 3D generation and novel-view synthesis.

16.7 Responsible use

Generative technology can create deceptive imagery (deepfakes), reproduce copyrighted or private data, and embed social bias. Learn about watermarking and provenance (C2PA), obtain proper licences for data and models, and consider the ethical implications of what you build (Chapter 18).

16.8 Neural rendering and 3D (awareness)

NeRF represents a scene as a neural network mapping 3D position and view direction to colour and density; volume rendering produces novel views. **3D Gaussian Splatting** represents scenes as millions of 3D Gaussians and renders in real time, quickly becoming the practical standard. Related areas: monocular **depth estimation** (MiDaS, Depth Anything), 3D reconstruction, and point-cloud networks (PointNet).

EXERCISE - Chapter 16 exercises

• Train an autoencoder on MNIST; use reconstruction error for anomaly detection by feeding it Fashion-MNIST images.

• Train a VAE and sample new digits; interpolate between two latent codes and view the morph.

• Train a small DCGAN on a simple dataset and observe mode collapse; then try WGAN-GP.

• Implement a tiny diffusion model (the training step above, plus a sampling loop) on MNIST.

• Fine-tune a Stable Diffusion LoRA on 20 images of a subject of your choice.

CHAPTER 17

Advanced Vision Tasks and Applications

Beyond classification, detection and segmentation lie many specialised tasks. An expert must recognise each one, know the standard approach, and know the tools. This chapter surveys them with working starting points.

17.1 Keypoint detection and pose estimation

Human pose estimation locates body joints (shoulders, elbows, knees), usually the 17 COCO keypoints. Approaches: **top-down** (detect people, then estimate keypoints in each box: HRNet, ViTPose) and **bottom-up** (detect all keypoints, then group them: OpenPose, HigherHRNet). Output is either **heatmaps** (a Gaussian peak per keypoint, trained with MSE) or direct regression. Metric: **OKS-based AP** (object keypoint similarity). Applications: sports analytics, rehabilitation, animation, fitness apps, gesture control. Tools: Ultralytics YOLO-pose, MediaPipe, MMPose.

```
from ultralytics import YOLO
model = YOLO("yolo11n-pose.pt")
res = model("people.jpg")[0]
print(res.keypoints.xy.shape)        # (num_people, 17, 2)
```

Related: hand pose, face landmarks, animal pose, 3D human mesh recovery (SMPL), action recognition from skeleton sequences.

17.2 Face analysis

Pipeline: **detect** (RetinaFace, YOLO-face, MTCNN), **align** using landmarks, **embed** with a network trained with margin-based losses (ArcFace, CosFace) so that same-identity faces cluster tightly on a hypersphere, then **compare** embeddings by cosine distance. Verification compares two faces; identification searches a gallery. Further tasks: attribute and emotion estimation, liveness / anti-spoofing.

> **WARNING - Ethics of face recognition**
>
> Face recognition raises serious privacy, consent, bias and legal concerns, and error rates differ across demographic groups. Many jurisdictions regulate or ban certain uses. Learn the law and the evaluation protocols (for example, per-group error rates) before building anything of this kind.

17.3 Optical character recognition and document understanding

OCR has two parts: **text detection** (find text regions: DBNet, CRAFT) and **text recognition** (read the characters: CRNN with **CTC loss**, or transformer-based TrOCR). End-to-end systems: Tesseract, EasyOCR, PaddleOCR, docTR. Document AI goes further, understanding layout, tables and forms: LayoutLM family, Donut, and modern vision-language models. Preprocessing from Chapter 10 (deskewing, adaptive thresholding, perspective correction) still improves accuracy considerably.

```
import easyocr
reader = easyocr.Reader(["en"])
for box, text, conf in reader.readtext("receipt.jpg"):
    print(f"{conf:.2f}  {text}")
```

17.4 Video understanding

Video adds the time dimension. Tasks: **action recognition**, temporal action detection, video captioning, video object segmentation, and video question answering.

- **2D CNN + temporal pooling** (simple baseline), **two-stream networks** (RGB plus optical flow).
- **3D CNNs:** C3D, I3D, SlowFast (a slow pathway for semantics and a fast pathway for motion).
- **Video transformers:** TimeSformer, VideoMAE, ViViT, Video Swin.
- Key challenges: heavy compute (sample frames sparsely, e.g. 8 to 32 per clip), long-range context, and dataset biases (models often recognise the scene instead of the action). Datasets: Kinetics, Something-Something, UCF101, AVA.

17.5 Depth estimation, 3D vision and SLAM

- **Monocular depth estimation:** predict depth from one image (MiDaS, DPT, Depth Anything); relative versus metric depth.
- **Stereo matching:** from two views via disparity, classical (SGBM) or learnt (RAFT-Stereo).
- **Point clouds (LiDAR):** PointNet, PointNet++, VoxelNet, PointPillars, point transformers. Essential in autonomous driving with 3D detection.
- **Multi-view geometry:** structure from motion (COLMAP), visual SLAM (ORB-SLAM3), visual odometry.
- **3D reconstruction and rendering:** NeRF and Gaussian Splatting (Chapter 16).
- **6-DoF object pose estimation:** for robotic grasping.

17.6 Anomaly detection and industrial inspection

When defects are rare and varied, train on **normal** images only and flag deviations. Methods: autoencoder reconstruction error, **PatchCore** (memory bank of normal patch features from a pretrained network), **PaDiM**, **FastFlow**, and student-teacher networks. Benchmark: MVTec AD. Library: `anomalib`. Metrics: image-level and pixel-level AUROC, and attention to the false-positive rate on the production line.

17.7 Medical imaging

Modalities: X-ray, CT, MRI, ultrasound, pathology slides, retinal photographs. Tasks: classification (disease), detection (nodules), segmentation (organs, tumours; **nnU-Net**), registration, report generation. Special concerns:

- **Data formats:** DICOM, NIfTI; handle intensity windowing and 3D volumes.
- **Small, imbalanced datasets:** transfer learning, strong augmentation, Dice losses.
- **Patient-level splits** to avoid leakage; external validation at other hospitals.
- **Calibration, uncertainty and explainability** for clinical trust; regulatory approval (FDA, CE).
- **Privacy:** de-identification; federated learning.

17.8 Remote sensing and geospatial vision

Satellite and aerial imagery: land-cover classification, building and road extraction, crop monitoring, change detection, disaster response, ship and vehicle detection. Particularities: multi-spectral bands (more than 3 channels; modify the first convolution), enormous images (tile with overlap), oriented bounding boxes, geographic coordinate systems, and strong domain shift between sensors and regions. Tools: `rasterio`, `torchgeo`, Google Earth Engine.

17.9 Image retrieval, similarity and metric learning

Learn an embedding space where similar images are close. Losses: **contrastive**, **triplet** (`max(0, d(a, p) - d(a, n) + margin)`), ArcFace, supervised contrastive. Retrieve with approximate nearest neighbour search (**FAISS**, ScaNN, HNSW). Applications: visual search in e-commerce, duplicate detection, face recognition, re-identification, place recognition. Metrics: Recall@K, mAP.

17.10 Image restoration and enhancement

Denoising, deblurring, super-resolution, low-light enhancement, dehazing, deraining, HDR. Architectures: DnCNN, NAFNet, Restormer, SwinIR, and diffusion-based methods. Metrics: PSNR and SSIM (pixel fidelity), LPIPS (perceptual).

```
def psnr(a, b, max_val=255.0):
    mse = np.mean((a.astype(np.float64) - b.astype(np.float64)) ** 2)
    return 10 * np.log10(max_val ** 2 / mse)
```

17.11 Robotics and embodied vision

Perception for grasping (segmentation, 6-DoF pose), visual servoing, navigation (SLAM, depth), and **vision-language-action models** that map images and language instructions to robot actions. Frameworks: ROS 2, Isaac Sim, MuJoCo, PyBullet. Real-time constraints and sensor fusion (camera, LiDAR, IMU) dominate engineering.

17.12 Autonomous driving perception stack

Camera-based detection and tracking, lane and drivable-area segmentation, traffic sign and light recognition, depth estimation, bird's-eye-view (BEV) representations (BEVFormer, LSS), LiDAR 3D detection, sensor fusion, occupancy prediction, and trajectory forecasting. Datasets: KITTI, nuScenes, Waymo Open, BDD100K. Safety, robustness to weather and rare events, and verification are as important as accuracy.

17.13 Adversarial robustness and security

Small, imperceptible perturbations can fool a classifier (**adversarial examples**). The **Fast Gradient Sign Method (FGSM)** creates one in a single step: `x_adv = x + epsilon * sign(grad_x Loss)`.

```
def fgsm(model, x, y, eps=0.03):
    x = x.clone().requires_grad_(True)
    loss = F.cross_entropy(model(x), y)
    loss.backward()
    return (x + eps * x.grad.sign()).clamp(0, 1).detach()
```

Defences: **adversarial training**, input preprocessing, ensembles, certified robustness. Related threats: data poisoning, backdoors, model stealing, privacy attacks (membership inference). Also evaluate robustness to natural corruptions (noise, blur, weather) and distribution shift, and use **out-of-distribution detection** to flag unfamiliar inputs.

EXERCISE - Chapter 17 exercises

- Use a pose model to count repetitions of an exercise (e.g. squats) from the knee angle over time in a video.
- Build a receipt reader: OCR, then parse the total with regular expressions.
- Train PatchCore (anomalib) on MVTec AD for one category and visualise anomaly maps.
- Create an image retrieval engine and report Recall@1 and Recall@5.
- Craft FGSM adversarial examples for a CIFAR-10 model and plot accuracy versus epsilon; then try adversarial training.

CHAPTER 18

Data, Experimentation, Deployment and Engineering Practice

In industry, the model is typically 20 percent of the work. Data quality, rigorous experimentation, and reliable deployment decide whether a project works in the real world. This chapter makes you a complete engineer.

18.1 Data-centric AI

Improving data usually beats improving architecture. A data-centric workflow:

1. **Define the task and the labelling guidelines precisely.** Ambiguous classes create label noise. Write down edge-case rules with pictures.
2. **Collect diverse data** reflecting real deployment conditions: lighting, devices, locations, seasons, demographics.
3. **Audit the data:** class counts, image sizes, duplicates (perceptual hashing, embedding similarity), corrupt files, near-duplicates across splits.
4. **Annotate with quality control:** multiple annotators, agreement measures (Cohen's kappa, box IoU between annotators), review rounds, tools such as CVAT, Label Studio and Roboflow. Use model-assisted pre-labelling (SAM, detectors) and correct the output.
5. **Find label errors** with tools like `cleanlab` (training loss and confident disagreement highlight suspicious labels).
6. **Active learning:** label the examples the model is most uncertain about, instead of random ones.
7. **Version your data** (DVC, Hugging Face datasets, lakeFS) as carefully as code.

Handling small datasets

Transfer learning, strong augmentation, self-supervised or foundation-model features, synthetic data, semi-supervised learning (pseudo-labelling, FixMatch), few-shot methods, and simply labelling more of the right data.

Avoiding bias and ensuring fairness

Measure performance **per subgroup** (lighting, skin tone, geography, camera model, age). Document datasets with datasheets and models with **model cards**. Recognise that uneven data representation produces uneven accuracy.

18.2 Experiment management

- Put everything in config files (YAML, Hydra) and record the git commit hash with every run.
- Log metrics, hyperparameters, samples of predictions and system info to **Weights & Biases**, **MLflow** or **TensorBoard**.

- Fix seeds, and repeat important runs with several seeds.
- Use **ablation studies**: remove or alter one component at a time to prove it matters.
- Keep a baseline: always compare with the simplest reasonable approach (a pretrained ResNet fine-tuned for 10 epochs).
- Maintain an experiment log (hypothesis, change, result, conclusion). This habit builds research maturity.

```
import wandb
wandb.init(project="cv-course", config={"lr": 1e-3, "model": "resnet18", "epochs": 20})
for epoch in range(20):
    wandb.log({"train_loss": tl, "val_acc": va, "lr": opt.param_groups[0]["lr"]})
```

18.3 Error analysis

After each training run, study mistakes systematically: gather misclassified or poorly detected samples, tag each by cause (blur, occlusion, unusual angle, label error, rare class), count tags, and fix the biggest group first. A confusion matrix, per-class metrics, and a gallery of worst-loss examples are your best tools. This loop, error analysis then targeted data or model change, drives most real-world improvement.

18.4 Scaling training: multiple GPUs and efficiency

- **DataParallel / DistributedDataParallel (DDP):** replicate the model on each GPU and split the batch; launch with `torchrun --nproc_per_node=4 train.py`. Prefer DDP over DataParallel.
- **Mixed precision (AMP, bfloat16)**, `torch.compile` for speed.
- **Gradient accumulation** to emulate large batches; **gradient checkpointing** to save memory.
- **Fully Sharded Data Parallel (FSDP)** and DeepSpeed for very large models.
- Efficient data loading: sufficient `num_workers`, fast storage (WebDataset, LMDB), decoding with GPU libraries (DALI), caching preprocessed data.
- **Profile** before optimising: `torch.profiler`, and watch GPU utilisation (`nvidia-smi`). Often the data loader, not the model, is the bottleneck.

18.5 Hugging Face and the open ecosystem

The Hugging Face hub hosts thousands of pretrained models and datasets; the `transformers`, `datasets`, `diffusers`, `timm` and `accelerate` libraries provide uniform interfaces.

```
from transformers import pipeline
detector = pipeline("object-detection", model="facebook/detr-resnet-50")
print(detector("street.jpg"))
classifier = pipeline("image-classification", model="google/vit-base-patch16-224")
print(classifier("cat.jpg")[:3])
```

Read model cards and licences carefully before using a model commercially.

18.6 Preparing a model for deployment

Export formats:

```
# TorchScript
scripted = torch.jit.trace(model.eval().cpu(), torch.randn(1, 3, 224, 224))
```

```
scripted.save("model.ts")

# ONNX: portable across runtimes
torch.onnx.export(model.eval().cpu(), torch.randn(1, 3, 224, 224), "model.onnx",
                  input_names=["image"], output_names=["logits"],
                  dynamic_axes={"image": {0: "batch"}, "logits": {0: "batch"}}, opset_version=17)

# run with ONNX Runtime
import onnxruntime as ort
sess = ort.InferenceSession("model.onnx", providers=["CPUExecutionProvider"])
logits = sess.run(None, {"image": np.random.rand(1, 3, 224, 224).astype(np.float32)})[0]
```

Optimisation: TensorRT (NVIDIA GPUs), OpenVINO (Intel), Core ML (Apple), TFLite (mobile, microcontrollers), ONNX Runtime; **quantisation** (FP16, INT8), **pruning** and **distillation** (Chapter 12). Always re-validate accuracy after optimisation, and benchmark latency and throughput on the actual target hardware with realistic batch sizes.

Pre/post-processing parity

A frequent production bug: training preprocessing differs from serving preprocessing (RGB versus BGR, interpolation method, normalisation constants, resize policy). Write preprocessing **once** and share it, and test that exported models give the same outputs as the original on sample images.

18.7 Serving models

```
# app.py: a minimal REST API with FastAPI
from fastapi import FastAPI, UploadFile
from PIL import Image
import io, torch

app = FastAPI()
model = torch.jit.load("model.ts").eval()
CLASSES = ["cat", "dog", "bird"]

@app.post("/predict")
async def predict(file: UploadFile):
    img = Image.open(io.BytesIO(await file.read())).convert("RGB").resize((224, 224))
    x = torch.from_numpy(np.array(img)).permute(2, 0, 1).float().div(255).unsqueeze(0)
    x = (x - MEAN) / STD                        # same normalisation as training
    with torch.no_grad():
        p = model(x).softmax(1)[0]
    i = int(p.argmax())
    return {"label": CLASSES[i], "confidence": float(p[i])}
# run: uvicorn app:app --host 0.0.0.0 --port 8000
```

Demonstrate quickly with **Gradio** or **Streamlit** (a UI in ten lines). Containerise with **Docker** so the environment is reproducible:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

For scale: batching requests, model servers (NVIDIA Triton, TorchServe, BentoML), autoscaling on Kubernetes, queues for asynchronous jobs, and GPU sharing.

18.8 Edge and mobile deployment

Edge devices (phones, Jetson boards, Raspberry Pi, cameras) offer low latency and privacy but limited compute. Choose small architectures (MobileNet, EfficientNet-Lite, YOLO-nano), quantise to INT8, use hardware accelerators (NPU, TPU, GPU), and measure power and thermal behaviour. Pipelines in C++ or with OpenCV DNN, TFLite, NCNN or ONNX Runtime Mobile.

18.9 Monitoring and maintenance (MLOps)

Models degrade as the world changes (**data drift** and **concept drift**). In production: log inputs and predictions (respecting privacy), monitor confidence distributions and input statistics, sample outputs for human review, collect hard cases and feed them back into training (the **data flywheel**), schedule retraining, keep rollback options, and use A/B or shadow testing before replacing a model. CI/CD for ML runs tests on data, code and model quality gates.

18.10 Testing machine learning code

- Unit-test data transforms (shape, range, dtype, label alignment after augmentation).
- Test that the model overfits a tiny batch.
- Test invariances (a horizontal flip of a cat image should still give "cat").
- Test serialisation: reload, same output.
- Build **slice-based tests**: required accuracy for each important subgroup, and a "golden set" of regression cases.

18.11 Ethics, privacy and responsibility

- **Privacy:** minimise personal data, anonymise (blur faces and licence plates), comply with laws such as GDPR; document consent.
- **Bias and fairness:** measure subgroup disparities; involve affected communities.
- **Safety and misuse:** consider dual-use risks, human oversight for high-stakes decisions.
- **Transparency:** publish model cards, limitations and intended use.
- **Environmental cost:** training uses energy; reuse pretrained models, avoid wasteful sweeps, report compute.
- **Intellectual property and licensing** of datasets, models and generated content.

EXERCISE - Chapter 18 exercises

• Audit a public dataset: find duplicates, label errors and class imbalance; write a short report.

• Wrap your best classifier in a FastAPI service, containerise it with Docker, and call it with `curl`.

• Export the model to ONNX; verify identical outputs; measure speed in PyTorch versus ONNX Runtime versus INT8.

• Set up Weights & Biases logging and run a learning-rate sweep.

• Write a model card for one of your projects, including limitations and subgroup performance.

CHAPTER 19

Becoming an Expert: Projects, Research Skills and Career

Knowledge becomes expertise through deliberate practice, original work, and contact with the research frontier. This final chapter gives you a project ladder, a method for reading and reproducing papers, a month-by-month schedule, and guidance for careers in industry and research, including graduate study.

19.1 The project ladder

Complete these projects in order. Each one trains a specific skill. Put every project on GitHub with a clear README: problem, data, method, results table, failure cases, how to run.

Level	Project	Skills demonstrated
1	Handwritten digit and Fashion-MNIST classifiers (MLP then CNN)	PyTorch basics, training loop
2	CIFAR-10 CNN reaching 93 percent or more	Augmentation, schedules, BatchNorm, residual blocks
3	Custom image classifier with transfer learning (own photos, 3 to 10 classes)	Data collection, fine-tuning, error analysis
4	Web demo of project 3 (Gradio or FastAPI plus Docker)	Deployment
5	Object detector on a custom dataset (YOLO) with a video demo	Annotation, detection metrics, tracking
6	Semantic segmentation (U-Net with pretrained encoder) on a real dataset	Dense prediction, Dice/IoU
7	Image search engine using CLIP/DINOv2 embeddings and FAISS	Foundation models, retrieval
8	Reproduce one paper (for example ResNet on CIFAR, U-Net, ViT-small) with reported numbers	Research rigour
9	Capstone: an original end-to-end application in a domain you care about	Everything

Capstone project ideas

- **Agriculture:** crop disease detection from leaf photographs on a phone, running offline; build and publish a dataset of local crops.

- **Healthcare:** segmentation of lesions in public medical datasets, with calibration and uncertainty maps.
- **Wildlife and environment:** species classification and counting from camera traps or drone imagery.
- **Smart cities:** traffic counting, parking occupancy, or road-damage detection.
- **Accessibility:** a scene description and text-reading tool for visually impaired users.
- **Education and culture:** OCR and layout analysis for low-resource scripts and historical documents.
- **Industry:** defect detection on a manufacturing line with anomaly detection.

Choose a problem that you understand, where you can access or create data, and where a local solution has real value. Original, well-evaluated work in a specific domain stands out more than yet another cats-versus-dogs classifier.

A capstone checklist

1. Problem statement and success metric agreed before coding.
2. Dataset documented: source, licence, size, splits, annotation process.
3. Baseline (simple) and improved model, with an ablation table.
4. Evaluation on a held-out test set, per-class and per-subgroup results, confidence intervals over several seeds.
5. Error analysis with example failures.
6. Efficiency: parameters, FLOPs, latency on specified hardware.
7. Demo (app or video) and clean, reproducible repository (requirements, config, seed, scripts).
8. Short report in paper format (Introduction, Related Work, Method, Experiments, Conclusion, Limitations).

19.2 Competitions and community

- **Kaggle:** study winning solutions, join image competitions, learn ensembling, test-time augmentation, and pseudo-labelling.
- **Open source:** contribute to PyTorch ecosystem libraries (torchvision, timm, Ultralytics, MMDetection, Hugging Face); fix documentation first, then bugs.
- **Communities and courses:** the fast.ai forums, Hugging Face community, Papers with Code, and local study groups. Teach others: writing a blog post or giving a talk is the best test of understanding.

19.3 How to read a research paper

Read in passes:

1. **Pass 1 (5 minutes):** title, abstract, figures, conclusion. Decide whether it matters to you.
2. **Pass 2 (30 to 60 minutes):** read the whole paper; skip proofs; note the main idea, the method diagram, datasets, metrics, baselines and the key table.

3 **Pass 3 (hours):** attempt to re-derive or re-implement; question every assumption; check the ablations and appendix; read the code if available.

Questions to ask of every paper: What problem is solved, and why does it matter? What is the central idea, in one sentence? What are the baselines, and are they fair (same data, same compute)? What evidence supports the claim? What are the limitations that the authors do not emphasise? How could I use or extend it?

Sources: arXiv (cs.CV), Semantic Scholar, Papers with Code, Google Scholar alerts, conference proceedings (CVPR, ICCV, ECCV, NeurIPS, ICLR, ICML, WACV, MICCAI for medical), and community digests.

19.4 Essential papers (your reading list)

Area	Papers (read in this order)
Foundations	Backpropagation (Rumelhart 1986); LeNet (LeCun 1998); AlexNet (2012); Batch Normalization (2015); Adam (2014); Dropout (2014)
Architectures	VGG; GoogLeNet; ResNet; DenseNet; MobileNet; EfficientNet; ConvNeXt
Detection	R-CNN; Fast and Faster R-CNN; FPN; SSD; YOLO (v1 and later); RetinaNet (Focal Loss); FCOS; DETR; Deformable DETR
Segmentation	FCN; U-Net; DeepLab v3+; Mask R-CNN; SegFormer; Mask2Former; Segment Anything
Transformers	Attention Is All You Need; ViT; DeiT; Swin Transformer; MAE
Self-supervised and multimodal	SimCLR; MoCo; BYOL; DINO and DINOv2; CLIP; SigLIP; LLaVA
Generative	VAE; GAN; DCGAN; StyleGAN; DDPM; DDIM; Latent Diffusion; ControlNet; classifier-free guidance; flow matching
3D and geometry	NeRF; 3D Gaussian Splatting; PointNet; Depth Anything

19.5 Reproducing and extending a paper

1 Pick a short, influential paper with public code and modest compute needs.
2 **Re-implement from the paper first**, then compare with the official code; differences reveal hidden details.
3 Reproduce the headline numbers on a smaller scale, or on the same dataset, and document the gap honestly.
4 Run **ablations** that the paper did not; try the method on a new dataset.
5 Write up findings and publish the code. A careful reproduction is a valuable contribution, and a stepping stone to original research.

19.6 Doing original research

- **Find a gap:** read the "limitations" and "future work" of recent papers; look for settings where methods fail (low data, new domains, efficiency).
- **Form a hypothesis** you can test: "Technique X improves robustness to blur on dataset Y."
- **Design a fair experiment:** strong baselines, same budget, multiple seeds, appropriate metric, held-out test set that is touched once.
- **Analyse why it works,** not only that it works; include failure analysis.
- **Write clearly:** structure, honest limitations, reproducibility details (hyperparameters, seeds, code, data).
- **Submit** to workshops first, then to main conferences; seek feedback from advisors and peers.

> **TIP - Research habits that compound**
>
> Keep a research notebook; maintain a bibliography manager (Zotero); read two papers a week; implement one idea a month; discuss with others regularly; and learn LaTeX (Overleaf) early.

19.7 A twelve-month study schedule

Months	Focus	Milestones
1	Python fundamentals and intermediate Python (Chapters 1-3)	Three small programs; one OOP project on GitHub
2	NumPy, Matplotlib, Pandas; start linear algebra (Chapters 4-5)	Image manipulation project; 3Blue1Brown videos done
3	Calculus, probability; machine learning basics (Chapters 5-6)	scikit-learn digit classifier; Andrew Ng course started
4	Neural networks from scratch; PyTorch (Chapters 7-8)	NumPy MLP; MNIST in PyTorch
5	Training techniques; start OpenCV (Chapters 9-10)	CIFAR-10 at 90 percent or more; document scanner
6	CNNs, architectures, transfer learning (Chapters 11-12)	Custom image classifier deployed as a demo
7	Object detection (Chapter 13)	YOLO on a custom dataset
8	Segmentation (Chapter 14)	U-Net project
9	Transformers, SSL, foundation models (Chapter 15)	Image search engine; ViT fine-tuning
10	Generative models and advanced tasks (Chapters 16-17)	Diffusion/LoRA experiment; one advanced-task project
11	Engineering practice (Chapter 18); paper reproduction	Reproduced paper with write-up
12	Capstone and portfolio (Chapter 19)	Capstone published; portfolio site; applications submitted

Weekly rhythm: three days of study and exercises, two days building a project, one day reading papers or watching lectures, one day rest or review. Review old material with spaced repetition: revisit each chapter's exercises after a month.

19.8 Courses and resources to complement this book

- **Python:** official Python tutorial; *Automate the Boring Stuff*; Corey Schafer videos.
- **Mathematics:** 3Blue1Brown; Khan Academy; MIT 18.06 (Strang); *Mathematics for Machine Learning* (Deisenroth et al.).
- **Machine learning:** Andrew Ng's Machine Learning Specialization; *Hands-On Machine Learning* (Geron); StatQuest videos.
- **Deep learning:** fast.ai Practical Deep Learning; Karpathy's *Neural Networks: Zero to Hero*; *Dive into Deep Learning* (d2l.ai); *Deep Learning* (Goodfellow, Bengio, Courville).
- **Computer vision:** Stanford CS231n; Michigan EECS 498 (Johnson); *Computer Vision: Algorithms and Applications* (Szeliski); *Multiple View Geometry* (Hartley and Zisserman); OpenCV and PyTorch documentation.
- **Engineering:** *Designing Machine Learning Systems* (Huyen); Full Stack Deep Learning; Made With ML.

19.9 Career paths

- **Computer vision / ML engineer:** build and deploy models. Portfolio, strong coding, MLOps skills.
- **Research scientist / research engineer:** advance methods. Publications, mathematical maturity, reproducibility; usually a graduate degree for scientist roles.
- **Applied scientist in a domain** (medical imaging, remote sensing, agriculture, robotics): domain knowledge plus vision expertise.
- **Founder or independent developer:** productise a niche solution.
- **Academia:** PhD, postdoctoral work, teaching and mentoring.

Graduate study

A master's or PhD is the standard route for research positions. Prepare by: strong grades and mathematics, a reproducible research-style project or a paper (even a workshop paper), letters from people who have seen your research ability, and a clear, specific research statement describing which problems you want to work on and why. Contact potential supervisors with a concise email showing you have read their work and sketching how you could contribute. Look for fellowships and funded positions in your target countries. Strong programming and a visible GitHub record are a real advantage.

Interview preparation for industry

Expect: Python coding (data structures, NumPy), ML fundamentals (bias-variance, metrics, regularisation), deep learning (backpropagation, CNN shapes, BatchNorm, attention), computer vision specifics (IoU, NMS, mAP, anchor boxes, receptive field), a take-home or live modelling task, and system design (design a visual search or defect inspection system: data, model, metrics, latency, monitoring). Practise explaining your projects clearly, including what failed.

19.10 Common mistakes of learners

- **Tutorial hell:** watching without building. Always follow with your own project.
- **Skipping the basics:** shaky NumPy and shapes cause endless bugs later.
- **Chasing the newest model** instead of mastering fundamentals and evaluation.
- **Leaking test data** and fooling yourself with inflated numbers.
- **Ignoring data:** most failures are data problems.
- **Not reading code:** study the source of torchvision, timm and Ultralytics.
- **Working alone:** share your work, ask questions, give feedback.

19.11 A final word

Computer vision rewards patience. You will spend many hours confused by shapes, stubborn bugs and models that refuse to learn; every expert has been there. Each problem you solve makes the next one easier. Keep your curiosity, keep building, keep measuring honestly, and remember that the goal is not only to master a technology but to apply it to problems that matter to people. The field needs thoughtful builders from every country and background. The path in this book is complete; now walk it, one chapter and one project at a time.

Appendix A: Glossary

Term	Meaning
Activation function	Non-linear function applied to a neuron's output (ReLU, sigmoid, GELU)
Anchor box	Predefined box shape used by detectors as reference for predictions
Augmentation	Label-preserving transformation that creates extra training data
Backpropagation	Algorithm computing gradients by applying the chain rule backwards
Batch size	Number of samples processed per optimisation step
BatchNorm	Layer normalising activations over a batch
Bias (model)	Error from overly simple assumptions; or an additive parameter
Bounding box	Rectangle locating an object
Channel	One layer of an image or feature map (R, G, B, or learnt)
CNN	Convolutional neural network
Confusion matrix	Table of predicted versus true classes
Convolution	Sliding-window weighted sum with a shared kernel
Cross-entropy	Loss comparing predicted probabilities with true labels
Dice	Overlap metric 2TP/(2TP+FP+FN)
Dropout	Randomly zeroing activations during training
Embedding	Vector representation of an item
Epoch	One pass through the entire training set
Feature map	Output of a convolutional filter across the image
Fine-tuning	Continuing training of a pretrained model on a new task
FLOPs	Floating-point operations, a measure of compute
FPN	Feature Pyramid Network; multi-scale features
Gradient	Vector of partial derivatives; direction of steepest increase
Hyperparameter	Setting chosen before training (learning rate, batch size)
IoU	Intersection over Union of two regions
Kernel / filter	Small weight matrix used in convolution
Latent space	Compressed representation learnt by a model
Learning rate	Step size of the optimiser
Logits	Raw, unnormalised model outputs before softmax

Term	Meaning
Loss	Quantity minimised during training
mAP	Mean average precision across classes (and IoU thresholds)
NMS	Non-maximum suppression; removes duplicate detections
Overfitting	Memorising training data, failing on new data
Padding	Border added around an image before convolution
Pooling	Downsampling operation (max or average)
Receptive field	Input region influencing one unit
Regularisation	Techniques limiting model complexity
ResNet	Network with residual skip connections
Semantic segmentation	Pixel-wise class labelling
Softmax	Converts scores to probabilities summing to 1
Stride	Step size of a sliding window
Tensor	Multi-dimensional array
Transfer learning	Reusing knowledge from a pretrained model
Transformer	Architecture built on self-attention
Underfitting	Model too simple to capture patterns
ViT	Vision Transformer
Weight decay	L2 penalty on weights

Appendix B: Quick Reference

B.1 Image tensor conventions

Library	Layout	Channel order	Value range
NumPy / PIL / Matplotlib	(H, W, C)	RGB	0-255 uint8 or 0-1 float
OpenCV	(H, W, C)	BGR	0-255 uint8
PyTorch	(N, C, H, W)	RGB	float, usually normalised

B.2 Loss function chooser

Task	Output layer	Loss
Binary classification	1 logit	BCEWithLogitsLoss
Multi-class classification	K logits	CrossEntropyLoss
Multi-label classification	K logits	BCEWithLogitsLoss
Regression	1 or more values	MSELoss, L1Loss, SmoothL1Loss
Segmentation	(K, H, W) logits	CrossEntropy + Dice
Detection	boxes and classes	Classification (focal/CE) + box (L1, GIoU)
Embedding learning	vectors	Contrastive, triplet, ArcFace

B.3 The ten-step debugging checklist

1. Can the model overfit a single batch?
2. Are the images and labels correct when plotted together?
3. Is the preprocessing identical in training and inference?
4. Is the initial loss close to ln(K)?
5. Are `model.train()`, `model.eval()`, `zero_grad()` and `no_grad()` used correctly?
6. Is the learning rate sensible (try 3e-4 with AdamW)?
7. Are shapes and dtypes what you expect at every stage?
8. Is there data leakage between splits?
9. Do the metrics measure what you actually care about?
10. Did you look at the failures?

B.4 Library installation cheat-sheet

```
pip install numpy pandas matplotlib scikit-learn jupyterlab
pip install opencv-python pillow albumentations scikit-image
pip install torch torchvision torchaudio        # see pytorch.org for GPU builds
pip install timm ultralytics segmentation-models-pytorch
pip install transformers datasets diffusers accelerate
pip install onnx onnxruntime fastapi uvicorn gradio
pip install wandb tensorboard
```

B.5 Where to go next

Return to the exercises of each chapter, finish the project ladder of Chapter 19, and then pick a specialisation (medical imaging, autonomous systems, generative models, 3D vision, or efficient edge AI) and go deep. Read, build, measure, and share. Good luck on your journey.